Republic of Yemen
Ministry of Higher Education and
Scientific Research
University of Science and Technology
Faculty of Computing and Information Technology

Analytical Intelligence AI: Ubuntu Server Intelligence Log Collection and Analysis Tool

Amr Khaled AL-Awadhi	202210400203
Nezar Faris AL-Hammadi	202210101320
Haitham Adnan AL-Shawafi	202210102395
Abdulrahman Abdulmalek AL-Halqi	202210400025

Supervised by

Dr. Wedad AL-Sorori
Lecturer at the University of Science and Technology

A graduation project document submitted to the Department of Information Technology in partial
fulfillment of the requirements for Bachelor's degree in Cybersecurity

2025

Summary

In an era of escalating cyber threats, securing Linux-based server infrastructure has become a critical challenge. Traditional Intrusion Detection Systems (IDS) often struggle to detect novel, sophisticatd attacks, while standalone behavioral models may suffer from high false positive rates. This project, **Analytical Intelligence**, addresses these limitations by developing a hybrid security monitoring platform that integrates signature-based detection with behavioral analysis.

The primary objective was to design and implement a scalable, microservices-based system capable of real-time threat detection. The solution leverages **Suricata IDS** for identifying known attack signatures (e.g., DoS, Port Scans) and a custom **LSTM (Long Short-Term Memory)** neural network for detecting anomalous SSH authentication patterns (e.g., Brute Force, Credential Stuffing).

The system was built using a modern software stack including **Docker** for containerization, **FastAPI** for the backend, **PostgreSQL** for data persistence. Key results demonstrated the system's efficacy in detecting a wide range of attack vectors with high precision. The integration of a real-time dashboard and **Telegram** alerting ensures immediate incident response capabilities.

In conclusion, Analytical Intelligence provides a robust, modular, and cost-effective security solution, successfully bridging the gap between deterministic rule-based detection and adaptive AI-driven analysis. Recommendations for future work include expanding the model to analyze command-line history and integrating SIEM capabilities for broader network visibility.

Authorization

We authorize University of . Faculty of . to supply copies of our graduation project report to libraries, organizations or individuals on request.

Student Name	Student No.	Signature
Amr Khaled AL-Awadhi	202210400203	
Nezar Faris AL-Hammadi	202210101320	
Haitham Adnan AL-Shawafi	202210102395	
Abdulrahman Abdulmalek AL-Halqi	202210400025	

Date: .....

Dedication

We dedicate this work to our beloved families, whose unconditional love, endless support, and constant prayers have been our greatest strength and motivation.

To our esteemed supervisor, **Dr. Wedad AL-Sorori**, for her invaluable guidance, patience, and encouragement throughout this journey. Her mentorship has been instrumental in bringing this project to fruition.

And to everyone who has taught us, supported us, and believed in us to reach this milestone.

Acknowledgment

First and foremost, all praise and thanks are due to Allah, the Almighty, for His grace and blessings that enabled us to complete this work.

We would like to express our deepest gratitude and profound appreciation to our supervisor, **Dr. Wedad AL-Sorori**. Her continuous support, patience, and invaluable guidance throughout this project were the beacon that lit our path. Her expertise, insightful feedback, and encouragement were crucial in shaping this research and bringing it to a successful conclusion.

We also extend our sincere thanks to the **University of Science and Technology** and the **Faculty of Computing and Information Technology** for providing us with the resources, knowledge, and academic environment necessary for learning and innovation.

Our heartfelt thanks go to our families for their unceasing encouragement, sacrifices, and belief in us. You have been our pillars of strength. To our friends and colleagues who stood by us and offered their help and motivation, thank you.

Finally, we acknowledge and thank everyone who contributed, directly or indirectly, to the success of this project.

Supervisor Certification

I certify that the preparation of this project entitled "**Analytical Intelligence AI: Ubuntu Server Intelligence Log Collection and Analysis Tool**" prepared by the students:

- **Amr Khaled AL-Awadhi**
- **Nezar Faris AL-Hammadi**
- **Haitham Adnan AL-Shawafi**
- **Abdulrahman Abdulmalek AL-Halqi**

was made under my supervision at the **Computer Science** Department in partial fulfillment of the requirements for the Bachelor Degree in **Cybersecurity**.

Supervisor: Dr. Wedad AL-Sorori

Signature: _____

Date: _____

Examination Committee

Project Title: Analytical Intelligence AI: Ubuntu Server Intelligence Log Collection and Analysis Tool

Supervisor

Name	**Position**	**Signature**
Dr. Wedad AL-Sorori	Supervisor	_____

Examination Committee

Name	**Position**	**Signature**
_____	_____	_____
_____	_____	_____
_____	_____	_____

Department Head . . . . . .

Table of Contents

List of Figures

List of Tables

List of Listings

Chapter 1

Introduction

1.1 Overview

With the significant expansion we are witnessing in recent years due to the use of digital systems and a wide range of online services, the collection and analysis of system logs has become critically important to ensure information security, detect potential threats, and develop effective countermeasures. Logs are a rich source of information about activities occurring within systems and networks, as they record everything that happens in the system. However, the challenge lies in analyzing the massive volume of data quickly and efficiently to enable the early detection of attacks or abnormal behaviors.

a lot of companies tend to use some of the famous systems such as Splunk[1], Graylog[2], and other Security Information and Event Management (SIEM) platforms.

these tools are often expensive and complex for many organizations with limited budgets.

This is where our project idea, ”Analytical Intelligence” and we call it (AI), comes in — a simple yet intelligent tool for collecting and analyzing system logs, supported by artificial intelligence techniques to detect the most common anomalies and attacks, using flexible software technologies.

1.2 Problem Statement

Many small organizations lack systems for monitoring and analyzing logs to detect any problems they may encounter, especially on Ubuntu servers. This is due to the difficulty of managing these servers, often due to the high cost or technical complexity of the systems. This has led to numerous risks, including a limited ability to detect attacks at an early stage, difficulty tracking security incidents and analyzing their causes, and reliance on manual methods, which has slowed operations and increased the number of vulnerabilities that cause various problems.

Therefore, a software solution was needed that was easy to deploy, affordable, and supported intelligent log analysis.

1.3 Project Objectives

The primary goal of this project is to develop a simple and intelligent tool for collecting and analyzing Ubuntu server logs via a dashboard, using modern technologies and artificial intelligence to detect suspicious activity and generate reports. Sub-goals include designing an easy-to-use graphical interface for viewing and analyzing logs, supporting log collection from Ubuntu

[1] Splunk is a commercial platform for searching, monitoring, and analyzing machine-generated data via a web-style interface.

[2] Graylog is an open-source log management tool for collecting, indexing, and analyzing both structured and unstructured data.

servers, implementing artificial intelligence algorithms for automatic anomaly detection, and providing alerts when suspicious activity occurs.

1.4 Project Scope and Limitations

- This tool is specifically designed for Ubuntu server environments and is not intended to replace full-scale business SIEM platforms.
- The tool focuses on collecting authentication logs and network traffic that may indicate the most common attacks.
- Accuracy depends on the quality and quantity of available log data that it collected.
- Machine learning algorithms may occasionally produce false positives.
- The dashboard displays results and reports in XLSX and CSV formats.

1.5 Project Methodology

This project takes a systematic approach to designing, developing, and evaluating an intelligent, easy-to-use log analysis system for Ubuntu server environments. It aims to support security monitoring through intelligent anomaly detection in logs, leveraging locations that enhance alert accuracy, and artificial intelligence models that provide more accurate information.

The methodology consists of the following key phases:

1. **Requirement Analysis:** Identifying the requirements and needs of small and medium-sized organizations regarding log management, especially on Ubuntu-based servers.
2. **Log Collection Design:** Configure secure log collection agents (auth_collector and suricata_collector) to collect authentication logs and network traffic from Ubuntu Server environments.
3. **Log Preprocessing:** Filter, parse, and normalize collected logs into a structured format suitable for machine learning and analysis.
4. **Anomaly Detection Engine:** Implement dual detection engines: SSH LSTM model for authentication anomalies and Suricata IDS for network threat detection.
5. **Dashboard Interface:** Develop a simple and interactive web dashboard using FastAPI and Jinja2 templates to display real-time system status, alerts, and reports.
6. **Testing and Evaluation:** Experiment with the system and test its anomaly detection performance using real logs from ubuntu server to improve response.

1.6 Report Organization

This report is divided into seven chapters, each one will focus on a part of the project:

- **Chapter 1:** Introduces the overview, problem statement, objectives, methodology, and scope of the project.
- **Chapter 2:** Reviews related work, and tools relevant to log analysis and anomaly detection.
- **Chapter 3:** Describes the architecture, components of the system, and project models.
- **Chapter 4:** Explains the design of the system, how it works, and some project codes.
- **Chapter 5:** Details the implementation process and which tools are used.
- **Chapter 6:** Presents the experimental results and discusses the system performance and findings.
- **Chapter 7:** Conclusion and suggests improvements and potential future recommendations.

Chapter 2

Background and Literature Review

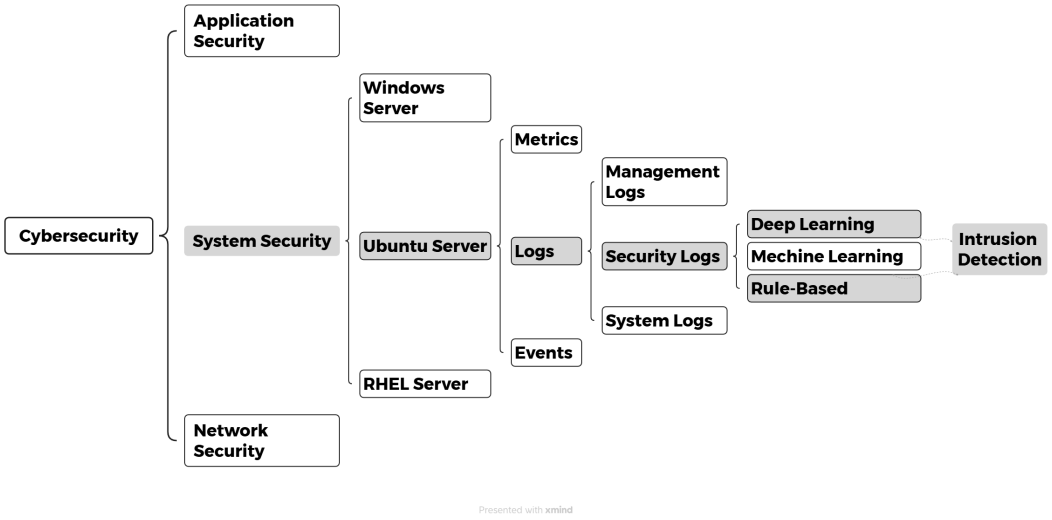


Figure 2.1: System Block Diagram

2.1 Background

2.1.1 Cybersecurity

Humanity today lives in an unprecedentedly interconnected era, where technology has become the backbone of daily life. It is no longer just an invention serving humanity; it is an indispensable infrastructure supporting several important sectors, such as communications, finance, corporate governance, and public services. However, this renaissance always comes with downsides and risks. Any flaw or vulnerability in digital systems can directly impact individuals, businesses, and even entire nations.

Therefore, cybersecurity has emerged as a fundamental response to modern stability, protecting data and digital assets from theft, tampering, and disruption. Societies unable to protect their infrastructure are at risk of collapse and social unrest. Today, cybersecurity is just as important as physical security for maintaining trust and enabling progress.

2.1.2 System Security

Cybersecurity is a comprehensive framework designed to protect organizations and individuals from various cyber threats. The increasing reliance on internet-connected devices has led to new challenges, including data leakage, hacking, and cyberattacks that attempt to exploit security vulnerabilities.

System security addresses these risks through several layers that contribute to mitigating vulnerabilities, such as network security, endpoint security, authentication and authorization mechanisms such as two-factor authentication or biometrics, security, and incident response. These measures ensure that systems remain robust against exploitation and unauthorized access.

2.1.3 Servers

Servers are the backbone of digital infrastructure, designed to provide high speed, resources, and shared services to multiple users simultaneously. Unlike personal computers, servers are optimized for stability, scalability, continuous operation, and high-volume operation. They host websites, databases, email systems, and cloud software.

Given their critical importance, servers are a prime target and frequent victim of cyberattacks. A single compromised server can allow attackers to access sensitive user data or control entire networks. There are different types of servers, such as:

- Windows Server
- Ubuntu Server
- RHEL Server, and a lot of different OS.

2.1.4 Ubuntu Server

Operating systems form the foundation of digital infrastructure, acting as an intermediary between devices and users. Any vulnerability in this layer could open the door to hackers.

Among the most widely used operating systems in enterprise environments are Ubuntu servers, known for their speed and high security. Servers host websites, databases, and cloud software, making them prime targets for cyberattacks. Therefore, server security includes:

- Regular fixes and updates.
- Strict access control, permission management, and privilege escalation.
- Firewall configuration and intrusion prevention.
- Continuous monitoring through Dashboard interface.

2.1.5 Logs

Logs are the living memory of any system, recording every event, transaction, and anomaly that occurs on the system. In Ubuntu Server environments, they include:

- Management logs.
- Security logs.
- System logs.
- Application logs.
- Network logs.

2.1.6 Security Logs

Security logs are a specialized category of log files that focus on events that impact system security. These logs include firewall activity, failed or repeated login attempts, privilege escalations, and any suspicious behavior that might indicate an attempted attack.

The main objectives of security logs are:

- **Threat Detection:** Identifying unauthorized access attempts and malicious activities.
- **Forensics:** It is considered a strong reference in the event of a crime.
- **Real-Time Monitoring:** Enabling administrators to receive alerts and respond to threats immediately.

2.1.7 Anomaly Detection

As organizations grow, the volume of stored logs increases, often reaching millions of events per day.

Manual review becomes impossible, leading to the adoption of advanced log management platforms such as Graylog and Splunk. These systems focus on log collection and issue real-time alerts for critical events.

The real challenge is distinguishing normal system behavior from malicious or suspicious patterns.

Anomaly detection addresses this by automatically identifying anomalies, such as:

- Rule-Based Detection.
- Machine Learning / Deep Learning
- Heuristic Analysis

2.1.8 Rule-Based

One of the most popular methods is rule-based, but it sometimes produces many false positives or fails to detect new attack methods.

Machine learning (ML) provides a more adaptive approach, learning the system's normal behavior and identifying events that may be deviant.

Deep learning and machine learning techniques used for anomaly detection include:

- Supervised learning algorithms (like classification models).
- Unsupervised learning methods (like Isolation Forest, K-Means).
- Deep learning models (like LSTM networks for time-series logs).

2.2 Literature Review

2.2.1 Introduction

The literature review aims to provide an overview of studies related to our system and previous approaches in the field of log analysis, anomaly detection, and methods, focusing on their methodologies, strengths, weaknesses, and the tools used. By analyzing this work, we can identify what has been achieved to date and the gaps that remain open for future research and development.

This review also discusses the similarities between the various literatures and compares them to our proposed system.

Finally, it highlights the research gap we seek to address through our project.

2.2.2 Zero-Day Exploit Detection in Network Flows

According to Touré et al. [1], a hybrid framework combining supervised classification, unsupervised clustering, and online learning was proposed to detect zero-day attacks.

Using datasets such as IBM and NSL-KDD, the system achieved high accuracy in identifying zero-day vulnerabilities, while minimizing false positives.

This study emphasizes the need to integrate diverse learning approaches, but also highlights challenges related to computational complexity and feature engineering.

2.2.3 Optimized Deep Isolation Forest (ODIF)

Gałka [2] presents ODIF, an improved version of Deep Isolation Forest, which reduces computational costs and memory usage while maintaining anomaly detection accuracy.

By simplifying data representation and eliminating duplicate processing, ODIF facilitates isolation-based anomaly detection in large-scale or resource-constrained environments.

This work demonstrates how efficiency improvements can enhance the applicability of AI methods in cybersecurity, but they are weak when the tree depth is large.

2.2.4 Collaborative Intrusion Detection Systems (CIDS)

Gómez et al. [3] present a framework for collaborative intrusion detection in log data, emphasizing decentralized cooperation across organizations and systems. The study categorizes detection approaches into sequential, embedding, and graph-based methods, and highlights the role of collaboration in reducing false positives. Their open-source evaluation platform provides a structured benchmark for testing algorithms, underlining the need for reproducibility and standardized assessment in log anomaly detection research.

2.2.5 ADALog: Adaptive Unsupervised Anomaly Detection in Logs

Zaremba et al. [4] propose ADALog, a self-attention–based framework that leverages a pre-trained DistilBERT model to detect anomalies in unstructured logs without requiring labeled data or parsing. The model achieves competitive performance on benchmark datasets while improving interpretability through heatmap visualization. However, its reliance on high computational resources and batch processing indicates the need for further optimization.

2.2.6 LogBERT: Log Anomaly Detection via BERT

Guo et al. [5] introduce LogBERT, a framework that applies BERT to system logs using self-supervised learning. The model captures contextual dependencies through tasks such as masked log key prediction and clustering of normal behavior. Experiments on datasets like HDFS and BGL show that LogBERT outperforms traditional and deep learning baselines. While effective, it relies on log parsers and hyperparameter tuning, which may limit adaptability in dynamic environments.

2.2.7 LAnoBERT: Log Anomaly Detection without Log Parsers

Lee, Kim, and Kang [6] extend LogBERT by removing the dependency on log parsers, enabling direct analysis of raw log data. Using BERT's masked language modeling, LAnoBERT improves flexibility and robustness across different log formats, achieving competitive performance compared to parser-based models. Its strengths lie in simplicity and adaptability, though it remains computationally demanding and less interpretable than other approaches.

2.2.8 Temporal Logical Attention Network (TLAN)

Liu et al. [7] propose TLAN, a model that captures temporal and logical relationships in log sequences using multi-scale feature extraction and attention mechanisms. Tested on distributed system logs, it improves detection of multi-step attacks such as SSH brute force followed by privilege escalation. Despite higher computational requirements, TLAN demonstrates the importance of modeling sequential dependencies for advanced log anomaly detection.

2.2.9 Semi-Supervised Framework for ALICE O^2

Krupa et al. [8] develop a semi-supervised framework for real-time anomaly detection in CERN's ALICE O^2 logs. By combining limited labeled anomalies with abundant normal data, the system balances supervised and unsupervised learning, improving detection accuracy and adaptability. This approach shows promise for large-scale infrastructures, though scalability and generalization remain key challenges for broader adoption.

2.2.10 Suricata Alert Detection Performance

Raharjo and Salman [9] investigated the performance of Suricata IDS when handling a large number of active Indicator of Compromise (IoC) rules. Their study revealed a critical trade-off: increasing the number of active rules significantly degrades detection accuracy. Specifically, they found that activating one million rules resulted in a 0% detection rate, highlighting the importance of rule optimization and selection in high-volume environments. This finding supports our project's approach of using a curated, specific ruleset rather than enabling all available rules blindly.

2.2.11 Effectiveness of Suricata-based IPS

Guo, Guo, and Zhang [10] demonstrated the high efficacy of rule-based systems when properly optimized. In their research on a Suricata-based Intrusion Prevention System (IPS), they achieved an impressive interception rate of 98.5% with a false interception rate near zero. Their work emphasizes that while rule-based systems differ from anomaly detection, their precision in handling known threats—when using high-performance, multi-threaded engines like Suricata—remains unmatched. This validates our project's reliance on Suricata for robust, signature-based implementation alongside AI-based analysis.

Title	Year	Authors	Methodology	Algorithm	Tools/Datasets	Advantages	Limitations
LogBERT: Log Anomaly Detection via BERT	2021	Haixuan Guo, Shuhan Yuan, Xintao Wu	Self-supervised framework using MLKP & VHM tasks	BERT	Drain, Loglizer, Logdeep; HDFS, BGL, Thunderbird	Captures both context; Self-supervised; Generalizes well; Open-source	Performance depends on hyperparameters; VHM less effective with long sequences
Zero-Day Exploit Detection in Network Flows	2024	Almamy Touré et al.	Hybrid: Supervised (CNN, DT, RF, KNN, NB) + Unsupervised (K-Means) + online learning	CNN, Decision Trees, Random Forest, KNN, Naïve Bayes, K-Means	IBM, NSL-KDD datasets	Detects unknown attacks; Adaptable via online learning	Needs representative data; Complex hybridization
Optimized Deep Isolation Forest (ODIF)	2025	Łukasz Gałka	Improves DIF with pre-sampling; reduces computational cost	ODIF	Comparisons with IF, DIF, ECOD, SGAE	Efficient for resource-limited environments; Scalable	Depends on representation quality; Limited on streaming data
Collaborative Anomaly Detection in Log Data	2025 (pre-proof)	André García Gómez et al.	Survey & taxonomy; open-source evaluation; tested with DeepLog	DeepLog, heuristic rules	Open-source; HDFS dataset	Benchmarking tool; Improves reproducibility	Sequential methods lose info; Graph methods are complex
ADALog: Unsupervised Anomaly Detection	2025	Przemek Pospieszny et al.	Unsupervised anomaly detection using DistilBERT +	DistilBERT, MLM	Python, PyTorch, Hugging Face	No parsing; Adaptive; High accuracy	High computational cost; Batch mode only; Not real-time

Figure 2.2: Literature Review

2.3 Tools

In recent years, researchers have developed specialized tools for log management and security analysis. These tools range from free to paid, each designed to meet organizations' needs in a way that best suits them. Most tools are designed for environments that handle massive amounts of data. This diversity is driven by the growing demand for flexible, scalable, and reliable cybersecurity solutions. Here, we review the most popular and widely used tools.

2.3.1 Graylog

Graylog [11] is an open-source log management platform that allows centralized log collection, storage, and analysis. It provides dashboards, search functionality, and real-time alerts, making it a great option for small and medium-sized organizations.

2.3.2 Splunk

Splunk [12] is a widely used enterprise-grade tool for real-time log analysis and SIEM. It excels at processing large datasets and offers advanced search and visualization features. However, it comes with significant licensing costs.

2.3.3 Elastic Stack (ELK)

The ELK stack [13] (Elasticsearch, Logstash, Kibana) is a flexible, open-source solution for log ingestion, indexing, and visualization. It's highly customizable, but requires technical expertise for deployment and maintenance.

2.3.4 LogBERT

LogBERT is a research-oriented framework that uses BERT-based models to detect anomalies in logs. It doesn't require labeled data and provides high accuracy, making it effective for server and operating system environments.

2.3.5 ADALog

ADALog uses DistilBERT for unsupervised anomaly detection in unstructured logs. It's accurate and interpretable, but it works in batch mode and needs high-performance hardware.

2.3.6 Sumo Logic

Sumo Logic [14] is a cloud-native log analytics platform that integrates machine learning for security monitoring and performance insights. It offers scalability and is ideal for cloud-based systems.

2.3.7 Datadog Log Management

Datadog [15] integrates log collection with infrastructure and performance metrics. It's effective for distributed systems and microservices, offering unified observability.

2.3.8 MobaXterm

MobaXterm [16] is an enhanced terminal application for Windows that provides a comprehensive toolbox for remote computing. It combines an X11 server, SSH client, and multiple network tools in a single portable executable. For security administrators, MobaXterm offers secure SSH/SFTP connections to Linux servers, multi-tab terminal sessions, and built-in tools for network diagnostics. It is particularly useful for managing Ubuntu servers remotely, allowing administrators to monitor logs, execute commands, and transfer files securely. The free version provides essential features suitable for small teams and academic environments.

Tool Name	Type	Primary Function	Strengths	Weaknesses	Use in Our Project	Applicability	Cost	Data Type
Graylog	Open Source	Log management and analysis	Easy UI, alerting, dashboards, flexible	Requires resources in large projects	Small/medium businesses	Easy on Ubuntu Server	Free	Unsupervised
Splunk	Commercial	Log analysis & SIEM	Powerful search, big data support	High cost, complex setup	High budget projects	Requires licenses	Paid	Hybrid
Elastic Stack (ELK)	Open Source	Collecting, processing, and visualizing logs	Flexible, scalable, strong integration	Complex to manage	Large-scale data	Multi-server deployment	Free (with paid options)	Unsupervised
LogBERT	Open Source (Research)	Anomaly detection in logs using BERT	High accuracy, self-learning	Sensitive to hyperparameters	Server monitoring	Easy via PyTorch	Free	Unsupervised
LAnoBERT	Open Source (Research)	Raw text analysis	High flexibility, fast deployment	High resource consumption	Fast-changing environments	Requires GPU	Free	Unsupervised
Sumo Logic	Commercial	Cloud-based log monitoring and analysis	Cloud integration, security insights	Dependent on internet and cloud services	Cloud-based projects	Easy via SaaS	Paid	Hybrid
Datadog Log Management	Commercial	Log and performance monitoring	Integrates with microservices	Cost complexity based on usage	Distributed systems	Easy in cloud environments	Paid	Hybrid

Figure 2.3: Literature Review

2.4 Gab Research

Despite advances presented in previous studies, ranging from hybrid zero-day vulnerability detection frameworks to transformer-based models such as LogBERT, LAnoBERT, and ADALog, several critical challenges remain that limit their practical adoption in small to medium-sized environments:

- **Complexity and Requirements:** Most modern approaches, particularly those utilizing deep learning transformers, require substantial computational power (GPUs) and complex infrastructure. This makes them prohibitively expensive and difficult to maintain for smaller organizations or standard Ubuntu servers.
- **Reliance on standard datasets only:** Despite achieving strong results on datasets such as HDFS and BGL, these models are rarely tested on live Ubuntu logs, weakening their reliability in practical use.
- **Deployment Difficulty:** Existing academic solutions often lack a unified, easy-to-deploy framework. They typically exist as isolated code repositories requiring extensive configuration, dependency management, and technical expertise to set up and integrate with live systems.
- **Lack of Focus on "Simplicity":** There is a scarcity of tools that prioritize "simplicity" without sacrificing essential security visibility. Most market solutions are either too Simple (basic logs) or too Complex (Enterprise SIEMs), leaving a gap for a lightweight, intelligent middle-ground solution.
- **Real-World Reliability:** Many proposed models are validated solely on static, standard datasets (HDFS, BGL) and struggle when applied to dynamic, real-world Ubuntu server logs which contain diverse and unstructured events.

2.4.1 Our Contribution

To address these gaps, **Analytical Intelligence (AI)** introduces a system designed specifically for **simplicity and efficiency**. Unlike complex SIEMs or heavy deep learning models, our project:

- **Ensures Ease of Deployment:** utilizing Docker and automated scripts to allow setup in minutes ("Simplicity of Loading").
- **Optimizes Resource Usage:** by using lightweight collectors and optimized detection logic suitable for standard servers.
- **Focuses on Practicality:** by providing a clear, direct dashboard that highlights actionable security insights for Ubuntu environments.

Chapter 3

Requirements Analysis and Modeling

3.1 Introduction

In this chapter, we outline the key requirements for an analytical intelligence (AI) tool, which combines speed and simplicity to collect, process, and analyze security logs from an Ubuntu server.

Unlike heavy-duty SIEM systems, this system focuses on clarity and low costs, making it suitable for small and medium-sized businesses. This chapter presents the foundations of design and modeling by defining the architecture, feasibility, methodology, and requirements. At the end of this chapter, we present models and diagrams (block diagram, use case, DFD, and database schema) to provide a clearer picture of the system and understand the requirements.

3.2 System Description

Analytical Intelligence (AI) is based on a multi-layered architecture focused on speed and ease of use. Specifically used for Ubuntu servers, each layer has a stage that explains a portion of the system's log lifecycle—from log collection, through filtering and cleaning, to detection/analysis, and finally, reporting. The layers are interconnected via flowcharts that illustrate the system's operation, allowing for continuous updates and work.

3.3 Feasibility Study

The feasibility of the system system will be evaluated through feasibility testing (technical, legal, operational and economic).

3.3.1 Technical Feasibility

The system relies on modern, widely adopted open source technologies, such as Ubuntu Server [17], Docker [18], FastAPI [19], PostgreSQL [20], and Suricata IDS [21]. These technologies and tools have proven their technical proficiency and robustness, making them reliable components that ensure stability, scalability, and reliability even under challenging conditions.

The system uses Python [22] for analysis, enrichment, and anomaly detection, making it flexible, given that it is one of the most widely used and powerful programming languages in the field of artificial intelligence. It also includes comprehensive libraries dedicated to data processing, deep learning [23], and cybersecurity.

By leveraging this technology stack, the system avoids the costs and limitations of proprietary solutions, while maintaining the adaptability needed to integrate additional modules in the future. Its robust architecture allows organizations to start with an easy setup, running on a single server, and scale to multiple servers as log volumes or analysis requirements increase.

3.3.2 Legal Feasibility

Legal feasibility is a critical factor for any cybersecurity system. Our project is designed with a strict ”Privacy-First” approach to ensure compliance with data protection laws and organizational security policies.

- **Data Isolation and Sovereignty:** A core legal advantage of our system is its architecture, which ensures complete data isolation. The system acts as a central analysis hub, but the data collected from each Ubuntu server is processed and stored with strict separation. There is no mixing of data between different monitored assets, ensuring that logs from one device never contaminate the records of another. This is crucial for forensic validity and legal accountability.
- **On-Premise Control:** Unlike cloud-based solutions where data leaves the organization's perimeter, our system is designed to run entirely within the user's controlled environment (On-Premise). This guarantees that sensitive logs—containing IP addresses, usernames, and system commands—remain under the organization's sole jurisdiction, complying with data sovereignty requirements.
- **Open Source Licensing:** The system is built on open-source components (MIT/Apache licenses), eliminating legal risks associated with proprietary software licensing and vendor lock-in.

3.3.3 Operational Feasibility

Operational feasibility is essential for a system, determining its effectiveness in real-world environments.

The platform is designed to be easy and practical, capable of operating efficiently even with medium-performance hardware. Its dashboards and analysis capabilities are simple and intuitive, enabling administrators and analysts to quickly interpret security events without the need for advanced security expertise.

This ease of use significantly reduces training and time requirements, especially for academic institutions and small and medium-sized businesses that may not have dedicated security teams.

We also offer real-time alerts, ensuring that incidents such as brute-force login attempts, privilege escalation, or suspicious IP addresses are detected and reported immediately to the appropriate personnel.

3.3.4 Economic Feasibility

Most importantly, it's economical. The system is designed to reduce overall costs while maintaining the option to expand when needed.

This solution relies on open source components with no licensing fees and can operate effectively on a single mid-range server in small or academic environments. Therefore, costs are concentrated on the one-time hardware acquisition and limited ongoing operations (electricity, storage, and simple administration time).

It's also supported by artificial intelligence, which reduces attack detection time and aids in rapid analysis of various logs.

Table 3.1: Economic Feasibility — Cost Table (Lean)

Item	Type	Frequency	Est. Cost (USD)
Ubuntu server (8 vCPU, 8 GB RAM)	CAPEX	One-time	—
SSD storage (512 GB)	CAPEX	One-time	30
Electricity	OPEX	Monthly	10
Internet	OPEX	Monthly	14
Team	OPEX	Monthly	500

3.4 Methodology

The system follows the Agile methodology. This approach allows the system to be implemented systematically while maintaining sufficient flexibility to accommodate improvements, feedback, and upcoming security challenges.

The process begins with the planning phase, where the overall project goals are defined and the key challenges of analyzing Ubuntu security logs are identified.

Following the planning phase, the requirements gathering phase is implemented, focusing on an in-depth analysis of security logs and the types of attacks they reveal. This phase leads to a comprehensive understanding of the logs and contributes to designing an architecture capable of handling them efficiently.

Then, the system design phase begins, producing architecture diagrams, workflows, and system models that formalize the layered architecture of AI. The focus is on developing each layer—collection, processing, detection, and presentation—independently without disrupting the entire system.

3.4.1 Data Collection Layer

The data collection layer is the foundation of the system, responsible for gathering real-time security data from Ubuntu servers. Data is collected from two primary sources:

- **Auth Logs (auth_collector):** Monitors `/var/log/auth.log` to capture failed login attempts, SSH activity, and sudo usage, providing essential insights into authentication security.

- **Network Traffic (Suricata IDS):** Inspects live network traffic using **Suricata**, a signature-based Intrusion Detection System. Suricata uses custom local rules to detect known attack patterns like DoS, port scans, and brute force attacks, producing alerts in `eve.json` format.

The system uses lightweight Python-based collector agents. The `auth_collector` tails log files, while the `suricata_collector` parses `eve.json` and ships alerts to the centralized backend via HTTP API calls with API key authentication.

3.4.2 The Processing Layer

Once data is collected, it moves to the processing layer for normalization and enrichment. The `auth_collector` sends raw authentication log lines to the backend, which parses, tokenizes, and normalizes them into structured events. Suricata alerts arrive pre-structured in JSON and are validated, deduplicated, and enriched with severity levels. All events are stored in PostgreSQL using JSONB payloads for flexible querying. This layer ensures consistent, high-quality data for downstream detection engines.

3.4.3 Detection and Analysis Layer

The detection and analysis layer transforms structured data into actionable security alerts using two complementary detection engines:

1. **SSH LSTM Model (Behavioral Detection):** An LSTM neural network analyzes sequences of authentication events to detect brute-force attacks and anomalous login patterns. It combines threshold-based rules (e.g., 5 failed logins in 5 minutes) with sequence-based deep learning predictions for comprehensive coverage.

2. **Suricata IDS (Signature-Based Detection):** The Suricata engine inspects network traffic against custom local rules configured for the deployment environment. It detects known attack patterns including DoS/DDoS, port scanning, and exploitation attempts.

This dual-engine approach balances **adaptability** (LSTM learns normal behavior) with **precision** (Suricata matches known signatures), providing comprehensive coverage of both known and emerging threats.

3.4.4 Presentation Layer

The presentation layer sits at the top of the system architecture and is designed to provide administrators and security analysts with a clear, intuitive, and actionable interface for system activity. A custom web dashboard built with Jinja2 templates and FastAPI is used to create interactive dashboards that transform log and detection data into clear insights. Dashboards present data

through visual elements such as severity badges, detection tables, and device statistics pages that track the evolution of events over time.

The presentation layer also supports the creation of exportable reports in formats such as XLSX and CSV. Combining ease of use with powerful analytical capabilities, this layer ensures that even users with limited technical expertise can easily monitor the system through an interface that provides the essential monitoring requirements. Visualization is handled by Jinja2 templates [24] and charts powered by Chart.js.

3.5 Dataset Description

Since we will be training, validating, and evaluating our anomaly detection model, we used the OpenSSH dataset provided by LogHub [25]. This dataset is a widely recognized benchmark for log analysis and anomaly detection research.

The OpenSSH dataset consists of over 600,000 log messages generated from real-world production servers. It captures a wide variety of SSH authentication events, including successful logins, failed password attempts, invalid user access, and session disconnects.

We selected this dataset for several reasons:

- **Relevance:** It exclusively focuses on SSH authentication logs, which directly aligns with our project's goal of detecting brute-force and unauthorized access attempts on Ubuntu servers [**?**].
- **Realism:** The logs are sourced from actual production environments, ensuring that our model is trained on realistic patterns rather than synthetic data.
- **Attack Patterns:** The dataset naturally contains real-world attack scenarios such as brute-force attacks, making it ideal for validating our anomaly detection logic.
- **Benchmarking:** As part of the LogHub collection [25], it allows our results to be compared against other state-of-the-art log analysis methods in the literature [5, **?**].
- **Availability:** It is publicly available and structured, facilitating integration into our machine learning pipeline using Python [22] and Scikit-learn [26].

By incorporating the OpenSSH dataset, we demonstrate the system's capability to learn from standard security logs and effectively detect authentication anomalies.

3.6 End-to-End Workflow

The project will be implemented in a structured and carefully planned manner to ensure quality and reliability of the system.

First, the critical Ubuntu security logs are enabled, validated, and synchronized to guarantee accurate event recording. Auth logs are collected by the `auth_collector` agent which tails `/var/log/auth.log`, while network traffic is monitored by **Suricata IDS** running in `af-packet` mode. Suricata alerts are shipped by the `suricata_collector` agent. These events are then parsed, normalized into JSON format, and enriched with contextual information such as severity levels to facilitate search, filtering, and correlation. The processed events are stored in PostgreSQL with JSONB payload fields to maintain fast and efficient queries.

After a short preparation period, the system establishes a baseline of what constitutes ”normal” activity. On top of this baseline, detection operates in two complementary ways: (1) rule-based mechanisms for well-defined patterns, such as repeated failed SSH login attempts within a short period, and (2) unsupervised anomaly detection models that highlight unusual behavior outside the norm. Each flagged event is assigned an anomaly score and a brief explanatory tag to support analyst understanding and reduce ambiguity.

Finally, all results are reviewed through an interactive dashboard that supports search, visualization, and forensic auditing. Reports summarizing security events are exported in standard formats (XLSX/CSV). Periodically, detection rules are updated, models are retrained to reflect new behavior, and indices are rotated or archived to keep the system efficient, secure, and sustainable over time.

3.7 Functional Requirements

3.7.1 Scope & Collection

The system’s functional requirements define the core capabilities the platform must provide to achieve its goals as an intelligent log analysis and security monitoring tool. Essentially, the AI must be able to collect a wide range of security logs from Ubuntu servers in real time, ensuring that critical events are captured without delay. These logs include authentication logs that detect failed login attempts and potential brute force attacks, logs that track user activity and activity, and firewall logs that provide visibility into blocked or suspicious network connections. The system standardizes these logs in a standardized structure—preferably JSON—for robust and seamless analysis. This requirement ensures consistency across diverse data formats and lays the foundation for rule-based and anomaly detection.

3.7.2 Detection & Analysis

Once the logs have been collected and normalized, AI must provide robust analysis capabilities. On one hand, it should employ rule-based detection mechanisms, where administrators define clear thresholds and conditions for generating alerts. For instance, multiple failed SSH login attempts within a short period should trigger a brute-force alert, while excessive or unusual

usage of the sudo command may indicate privilege escalation. On the other hand, the system should incorporate anomaly detection techniques powered by machine learning, which allow it to identify suspicious activities that do not match predefined rules. By establishing baselines of normal system behavior, the anomaly detection component can detect deviations such as unusual login times, unexpected process executions, or connections from atypical geographic locations. Together, these dual modes of detection provide comprehensive coverage of both known and emerging threats.

3.7.3 Real-Time Alerting

Another essential functional requirement of AI is the ability to generate real-time alerts. Detection is meaningless without rapid communication of results, and therefore the system must deliver timely notifications to security analysts or administrators through configurable channels such as dashboard alerts, emails, or integration with incident management platforms.

3.7.4 Visualization, Search, Forensics, and Reporting

In addition to alerts, AI should provide simple visualization capabilities. Through user-friendly dashboards built using FastAPI and Jinja2 templates, administrators should be able to visualize security events in graphical form. These dashboards transform complex log data into clear, actionable insights. Additionally, the system should support powerful search and query features that enable users to filter and investigate logs based on criteria such as date range, IP address, severity level, or event type. This functionality is critical for forensic investigations and reconstructing the sequence of events surrounding an incident. Finally, AI should support automated reporting by generating summaries in multiple formats, including CSV and XLSX, for compliance audits and corporate documentation.

3.8 Non-Functional Requirements

Non-functional requirements specify the quality attributes of the system, such as performance, reliability, and usability. These requirements ensure that the system is effective and sustainable in a real-world environment.

3.8.1 Performance

The system is designed to handle high volumes of log data efficiently. It is optimized to process logs from multiple sources with minimal latency, ensuring that alerts are generated in near real-time. The lightweight collector agents are designed to have a minimal footprint on the host servers, ensuring that security monitoring does not degrade the performance of the monitored systems.

3.8.2 Scalability

The modular architecture of the system allows for horizontal scalability. New server nodes can be added to the monitoring capability easily by deploying the collector agents. The backend is built to handle increasing loads, making the system suitable for small test environments as well as larger deployments.

3.8.3 Usability

A key focus of the project is ensuring the system is accessible to users with varying levels of technical expertise. The dashboard provides a clean, intuitive interface for visualizing data and managing alerts. Installation and deployment are simplified through the use of Docker and automated scripts, reducing the complexity of setting up the environment.

3.8.4 Reliability

The system ensures continuous operation through robust error handling and service management. The use of containerization helps in maintaining consistent environments and reduces the risk of dependency conflicts. In the event of a service restart, the system is designed to recover quickly and resume log processing without manual intervention.

3.8.5 Maintainability

The codebase is structured and modular, facilitating easy updates and maintenance. The use of standard technologies (Python, Docker, PostgreSQL) ensures that the system can be maintained and extended by developers with standard skill sets. Configuration is centralized, allowing for easy adjustments to detection rules and system settings.

3.9 System Modeling

This section presents the system models that formalize the architecture and design of AI.

3.9.1 Block Diagram

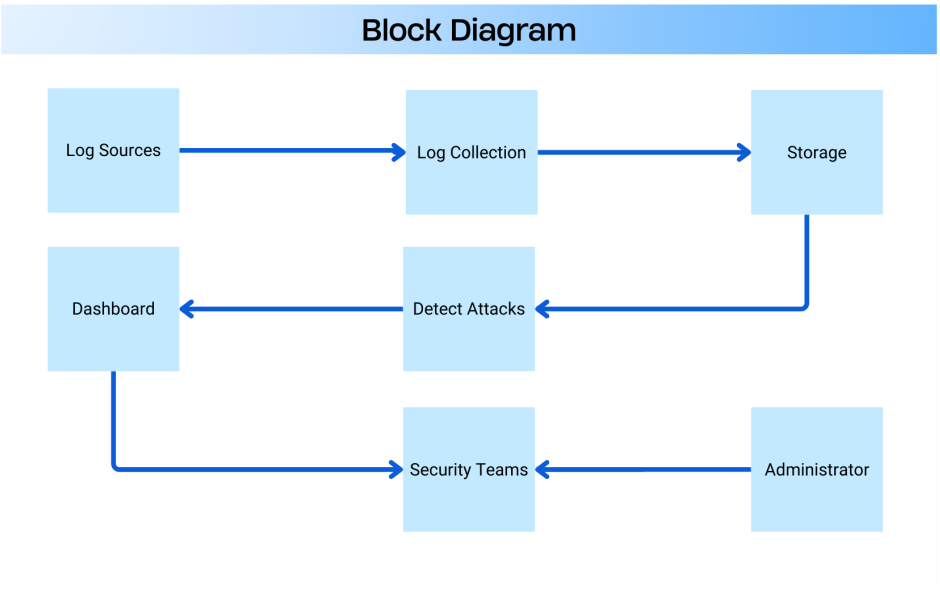


Figure 3.1: System Block Diagram

3.9.2 Use Case Model

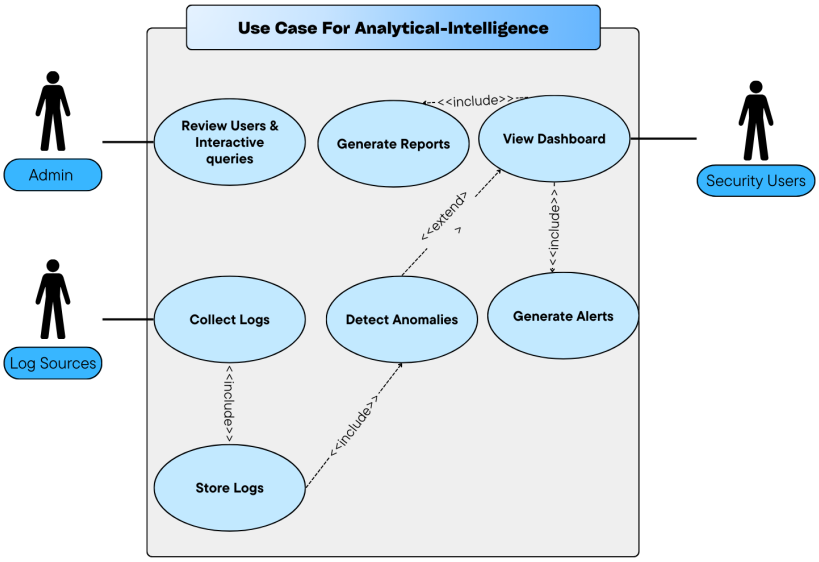


Figure 3.2: Use Case Diagram

3.9.3 DFD – Data Flow Diagrams

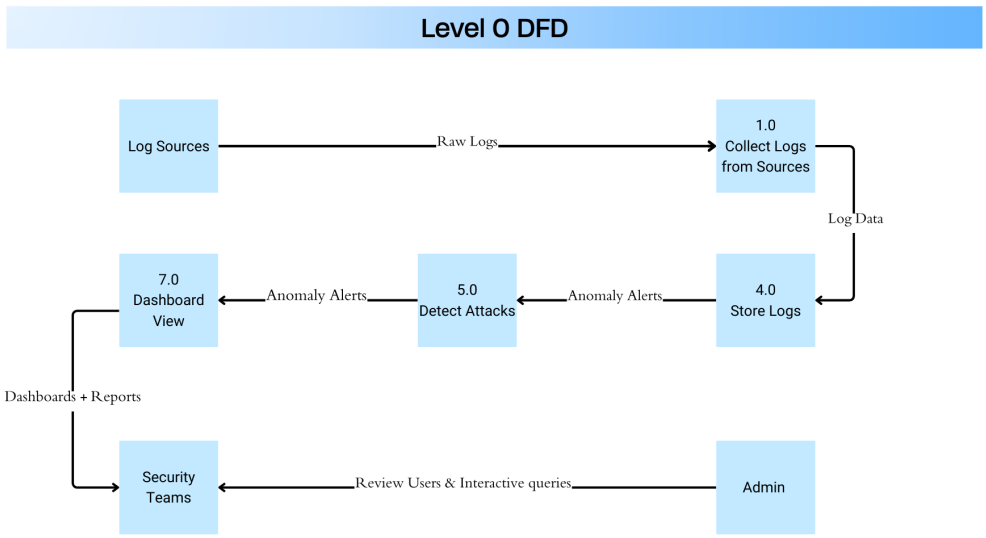


Figure 3.3: Data Flow Diagram

3.10 Database Schema

The system uses PostgreSQL as the relational database. PostgreSQL provides ACID compliance, robust querying capabilities, and supports JSONB for flexible storage of raw event payloads. The database schema consists of five main tables:

3.10.1 Devices Table

Stores registered sensor devices with approval workflow status.

```
CREATE TABLE devices (
device_id TEXT PRIMARY KEY,
hostname TEXT,
ip TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
last_seen TIMESTAMPTZ DEFAULT NOW(),
os TEXT,
role TEXT,
tags TEXT,
-- Device approval workflow
approval_status TEXT NOT NULL DEFAULT 'pending',
approved_at TIMESTAMPTZ NULL,
approved_by TEXT NULL,
blocked_reason TEXT NULL
```

```
);
```

Listing 3.1: Devices Table Schema

3.10.2 Raw Events Table

Stores all incoming events as-is with JSONB payloads for flexible querying.

```
CREATE TABLE raw_events (
id BIGSERIAL PRIMARY KEY,
ts TIMESTAMPTZ NOT NULL,
device_id TEXT REFERENCES devices(device_id),
event_type TEXT NOT NULL,  -- 'auth' or 'suricata'
payload JSONB NOT NULL
);
```

Listing 3.2: Raw Events Table Schema

3.10.3 Detections Table

Stores detection records from both SSH LSTM and Suricata with deduplication tracking.

```
CREATE TABLE detections (
id BIGSERIAL PRIMARY KEY,
ts TIMESTAMPTZ NOT NULL,
device_id TEXT REFERENCES devices(device_id),
raw_event_id BIGINT REFERENCES raw_events(id),
model_name TEXT NOT NULL,  -- 'ssh_lstm' or 'suricata'
label TEXT NOT NULL,        -- attack type
score DOUBLE PRECISION NOT NULL,
severity TEXT NOT NULL,     -- LOW, MEDIUM, HIGH, CRITICAL
details JSONB NOT NULL,
-- Deduplication and Traceability
occurrences BIGINT DEFAULT 1,
first_seen TIMESTAMPTZ DEFAULT NOW(),
last_seen TIMESTAMPTZ DEFAULT NOW(),
-- Network fields for optimization
src_ip TEXT,
dst_ip TEXT,
src_port INT,
dst_port INT,
proto TEXT
);
```

Listing 3.3: Detections Table Schema

3.10.4 Users Table

Stores UI authentication users with session management.

```
CREATE TABLE users (
id BIGSERIAL PRIMARY KEY,
username TEXT UNIQUE NOT NULL,
full_name TEXT NULL,
password_hash TEXT NOT NULL,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ DEFAULT NOW(),
last_login TIMESTAMPTZ NULL
);
```

Listing 3.4: Users Table Schema

3.10.5 User Login State Table

Tracks login attempts and lockout state for progressive lockout security.

```
CREATE TABLE user_login_state (
user_id BIGINT PRIMARY KEY REFERENCES users(id),
failed_count INT NOT NULL DEFAULT 0,
lock_level INT NOT NULL DEFAULT 0,  -- 0=none, 1=5min, 2=1h
lock_until TIMESTAMPTZ NULL,
last_failed TIMESTAMPTZ NULL,
updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

Listing 3.5: User Login State Table Schema

3.11 API Endpoints Summary

Table 3.2 summarizes the key API endpoints exposed by the backend. The API follows RESTful conventions with JSON responses.

Table 3.2: API Endpoints Summary

Endpoint	Method	Auth	Purpose
Ingestion Endpoints			
/api/v1/ingest/auth	POST	API Key	Ingest auth.log events
/api/v1/ingest/suricata	POST	API Key	Ingest Suricata alerts
System Endpoints			
/api/v1/health	GET	None	Health check with model status
/api/v1/stats	GET	Session	Dashboard statistics
/api/v1/recent-detections	GET	Session	Recent detections
/api/v1/analytics	GET	Session	Chart data for dashboard
Device Management			
/api/v1/devices	GET	Session	List all devices
/api/v1/devices/{id}/approve	POST	Session	Approve pending device
/api/v1/devices/{id}/block	POST	Session	Block device
Authentication			
/login	POST	None	UI login (session-based)
/logout	POST	Session	UI logout
Reports			
/api/v1/reports/export	GET	Session	Export CSV/XLSX reports

Ingestion Payload Schemas:

- **Auth Payload:** `device_id`, `hostname`, `device_ip`, `timestamp`, `line` (raw auth.log line)
- **Suricata Payload:** `device_id`, `hostname`, `device_ip`, `alert` (Suricata EVE JSON alert object)

Authentication Methods:

- **API Key:** Header `INGEST_API_KEY` for sensor-to-server communication
- **Session:** Cookie-based session after UI login for dashboard access

Chapter 4

Project Design

4.1 Introduction

This chapter presents the detailed design of the Analytical-Intelligence system, translating the requirements from Chapter 3 into concrete architectural decisions, component structures, and implementation patterns. The design emphasizes modularity, containerized deployment, and dual-model detection covering both authentication (SSH) and network traffic analysis.

4.2 Distributed System Architecture

4.2.1 Central AI Server (AI Central Brain)

The central server hosts the FastAPI backend with the following components:

- **Model Loader:** Loads SSH LSTM model at startup
- **Ingestion Router:** Handles `/api/v1/ingest/auth` and `/api/v1/ingest/suricata`
- **Detection Engine:** Runs SSH LSTM model server-side; Suricata alerts are ingested directly
- **Database Layer:** Async PostgreSQL via SQLAlchemy
- **UI Router:** Serves Jinja2-templated dashboard pages
- **Notification Bus:** Async queue for Telegram alerts

4.2.2 Unified Sensors and Intelligent Agents

The system employs a two-stack architecture separating data collection from analysis:

4.2.2.1 Network Level Monitoring (Suricata IDS)

The Suricata IDS monitors network traffic for threats:

- Runs in `af-packet` mode with `network_mode: host` for raw packet access
- Uses custom local rules for signature-based detection
- Produces structured alerts in EVE JSON format (`eve.json`)
- Detects DoS, DDoS, port scanning, and exploitation attempts

4.2.2.2 Operating System Level Monitoring (Auth Collector)

The auth collector agent monitors authentication events:

- Tails `/var/log/auth.log` using file inode tracking
- Filters SSH/PAM-related events
- Implements automatic reconnection on failures
- Maps events to the **SSH LSTM Model** pipeline

Table 4.1: Sensor to Model Mapping

Sensor	Data Source	Detection Model
auth_collector	/var/log/auth.log	SSH LSTM Model
suricata	Network interface (af-packet)	Suricata IDS (Signature)
suricata_collector	eve.json	Alert shipper to backend

4.2.3 Communication Protocol

All sensor-to-server communication uses HTTP POST with API key authentication:

- **Header:** `X-API-Key: <INGEST_API_KEY>`
- **Content-Type:** `application/json`
- **Retry Policy:** Exponential backoff on connection failures
- **Batching:** Configurable for Suricata via `eve.json` rotation

Auth Payload Schema:

```
{
    "device_id": "sensor-ubuntu-01",
    "hostname": "webserver-01",
    "device_ip": "192.168.1.100",
    "timestamp": "2025-01-20T10:30:00Z",
    "line": "Jan 20 10:30:00 sshd[1234]: Failed password..."
}
```

Listing 4.1: Auth Event Payload

Suricata Alert Payload Schema:

```
{
    "device_id": "sensor-ubuntu-01",
    "hostname": "webserver-01",
    "device_ip": "192.168.1.100",
    "alert": {
        "action": "allowed",
        "signature": "Potential SSH Scan",
        "signature_id": 1000001,
        "severity": 2,
        "src_ip": "10.0.0.5",
        "dest_ip": "192.168.1.100",
        "dest_port": 22
    }
}
```

Listing 4.2: Suricata Alert Payload

4.3 Containerization and Deployment Architecture

To ensure consistency, scalability, and ease of deployment, the entire system is designed around a containerized microservices architecture using Docker. This approach isolates dependencies, simplifies updates, and ensures that the system runs identically across different environments (Development, Testing, and Production).

4.3.1 Microservices Design

The system is decomposed into distinct services, each responsible for a specific function and running in its own isolated container:

1. **Backend Service (App):** The core FastAPI application acting as the central brain. It exposes REST APIs for log ingestion and serves the dashboard. It runs in a stateless container, allowing for horizontal scaling if needed.

2. **Database Service (DB):** A PostgreSQL container responsible for persistent storage of logs, alerts, and user data. It uses a named volume for data persistence.

3. **Suricata Service (Net-Sensor):** A specialized container running the Suricata IDS engine. It requires `host` networking mode or `CAP_NET_ADMIN` privileges to capture live network traffic from the host interface.

4. **Suricata Collector Service (Net-Shipper):** A committed agent that reads the `eve.json` output from the Suricata engine and ships alert data to the Backend API in real-time.

5. **Auth Collector Service (Log-Sensor):** A lightweight Python agent that mounts the host's `/var/log/auth.log` in read-only mode to detect authentication events.

6. **pgAdmin Service (Administration):** A web-based interface for managing the PostgreSQL database, used for debugging and manual data inspection.

4.3.2 Orchestration and Networking

Docker Compose is used to orchestrate these services, defining their networks, volumes, and runtime configurations in a clear, declarative format.

- **Internal Network:** A dedicated bridge network (`ai-network`) connects the Backend and Database, ensuring they can communicate securely without exposing the database port to the outside world.

- **Host Integration:** Collectors interact securely with the host system. The Auth Collector accesses specific log files via read-only bind mounts, ensuring it cannot tamper with system logs.

4.3.3 Deployment Configuration

The following listing illustrates the structural design of our `docker-compose.yml`, highlighting the service definitions and security constraints:

```
services:
  # Central Analysis Brain
  backend:
    image: analytical-ai/backend:latest
    ports: ["8000:8000"]
    networks: [internal_net]
    depends_on: [db]

  # Secure Storage
  db:
    image: postgres:15-alpine
    volumes: [db_data:/var/lib/postgresql/data]
    networks: [internal_net]

  # Database Administration UI
  pgadmin:
    image: dpage/pgadmin4:latest
    ports: ["5050:80"]
    networks: [internal_net]
```

```
    depends_on: [db]

  # Network Sensor (Suricata Engine)
  suricata:
    image: flowpro/suricata:latest
    network_mode: "host"  # Required for raw packet capture
    volumes:
      - ./config/suricata:/etc/suricata
      - ./logs:/var/log/suricata

  # Network Alert Shipper
  suricata_collector:
    build: ./collectors/suricata
    network_mode: "host"
    volumes:
      - ./logs:/var/log/suricata:ro # Read-Only access to logs
    environment:
      - BACKEND_URL=http://backend:8000

  # Auth Log Sensor
  auth_collector:
    build: ./collectors/auth
    volumes:
      - /var/log/auth.log:/var/log/auth.log:ro  # Read-Only access
    environment:
      - BACKEND_URL=http://backend:8000
```

Listing 4.3: Docker Compose Services Architecture

4.4 Data Processing and Storage Design

4.4.1 PostgreSQL Database Design

The database consists of three tables with foreign key relationships:

- **devices:** Sensor registry with device_id, hostname, IP, last_seen
- **raw_events:** All events stored with JSONB payload, event_type (auth/suricata)
- **detections:** Alert records with model_name, label, score, severity, details

4.4.2 Feature Engineering

4.4.2.1 SSH Model Features

The SSH LSTM model uses token sequences derived from auth.log parsing:

Table 4.2: SSH Token Vocabulary

Token	Pattern
FAILED_PASSWORD	Failed password for
INVALID_USER	Invalid user / Failed password for invalid user
ACCEPTED_PASSWORD	Accepted password for
ACCEPTED_PUBLICKEY	Accepted publickey for
PAM_AUTH_FAILURE	pam_unix.*authentication failure
DISCONNECT	Disconnected from
CONNECTION_CLOSED	Connection closed by

4.4.2.2 Suricata Alert Features

Suricata provides structured EVE JSON alerts with the following key fields:

- **Alert Metadata:** signature, signature_id, severity, category
- **Network Context:** src_ip, dest_ip, src_port, dest_port, protocol
- **Flow Context:** flow_id, pkts_toserver, pkts_toclient, bytes_toserver
- **Additional Context:** app_proto (http, dns, tls), tls_sni, http_hostname

4.5 Algorithm Design

4.5.1 Specialized Models (Server-Side)

4.5.1.1 SSH LSTM Model Design

The SSH model operates in dual detection mode:

1. **Threshold-Based Detection:**

- Tracks failed login attempts per source IP using `SSHEventTracker`
- Maintains rolling 1-hour window of failed attempts
- Triggers when count $\geq$ `SSH_BRUTEFORCE_THRESHOLD` (default: 9) within `SSH_BRUTEFORCE_WINDOW_SE` (default: 300)
- Labels: `SSH_BRUTE_FORCE`

2. Sequence-Based Detection (LSTM):

- Analyzes token sequences using LSTM neural network
- Window size: configurable (default: 10 tokens)
- Predicts anomaly score (0.0 to 1.0)
- Triggers when score $\geq$ model threshold
- Labels: `SSH_ANOMALY`

4.5.1.2 Suricata IDS Detection

Suricata uses signature-based detection with custom local rules:

Detection Pipeline:

1. Suricata inspects network traffic in real-time (af-packet mode)
2. Matches packets against local ruleset signatures
3. Generates structured alerts in EVE JSON format
4. `suricata_collector` ships alerts to backend API

Alert Processing:

- **Severity Mapping:** Suricata severity (1-3) mapped to system severity levels
- **Deduplication:** Same (src_ip, signature_id) within window aggregated
- **Category Filtering:** Configurable category allowlist

4.5.2 Decision Gating and Alert Quality Controls

The system implements multiple gating layers to ensure alert quality:

Table 4.3: Alert Quality Controls

Control	Configuration	Purpose
Deduplication Window	SURICATA_ROLLUP_WINDOW_MINUTES	Time bucket for alert aggregatio
SSH Threshold	SSH_BRUTEFORCE_THRESHOLD	Failed login count limit
SSH Window	SSH_BRUTEFORCE_WINDOW_SECONDS	Time window for brute force
Local Rule Range	SURICATA_LOCAL_SID_MIN/MAX	Enforce local AI rules only

4.5.3 Severity Assignment

Severity levels are assigned based on attack type and confidence:

- **CRITICAL:** DDoS with high confidence
- **HIGH:** DoS, SSH Brute Force with high failed count
- **MEDIUM:** Port Scanning, moderate confidence attacks
- **LOW:** Low confidence detections, SSH Anomaly

4.6 Dashboard User Interface Design

The web dashboard provides visibility into both SSH and Suricata detections:

Table 4.4: Dashboard Pages and Endpoints

Page	Route	Content
Dashboard	/	Statistics cards, recent detections
Alerts	/alerts	Filterable detection list (SSH + Suricata)
Auth Events	/events/auth	Raw auth.log entries
Suricata Alerts	/events/suricata	Suricata EVE alerts
Devices	/devices	Sensor inventory with alert counts
Models	/models	SSH model + Suricata status
Reports	/reports	Export interface (CSV/XLSX)

The main dashboard, illustrated in Figure 4.1, provides a centralized view of the system's security posture. It features real-time statistics cards summarizing total detections, active sensors, and recent alerts, offering security analysts an immediate understanding of the current threat landscape.

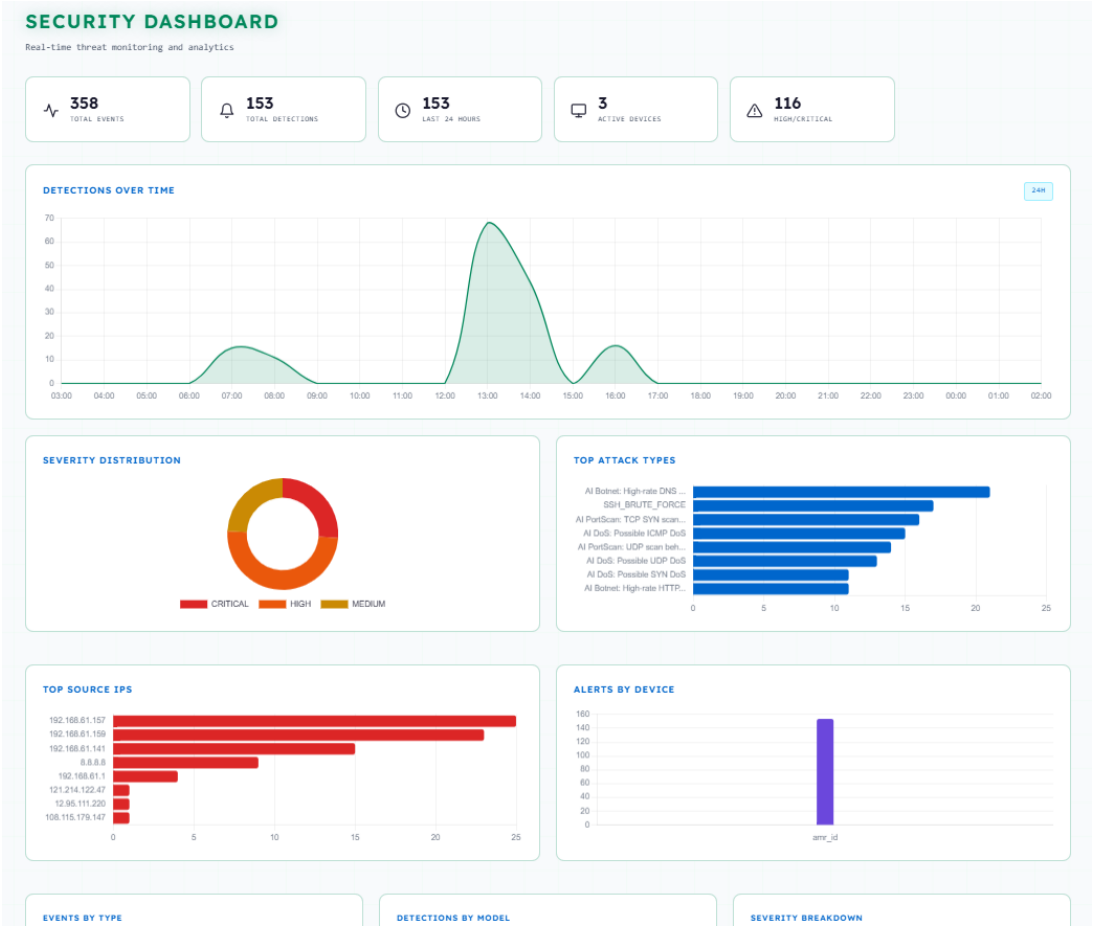


Figure 4.1: Main Dashboard View

Detailed management of security events is handled through the Alerts page (Figure 4.2). This interface allows administrators to view, filter, and analyze detections from both the SSH LSTM model and Suricata IDS. Each row displays critical metadata such as timestamp, severity, source IP, and detection type.

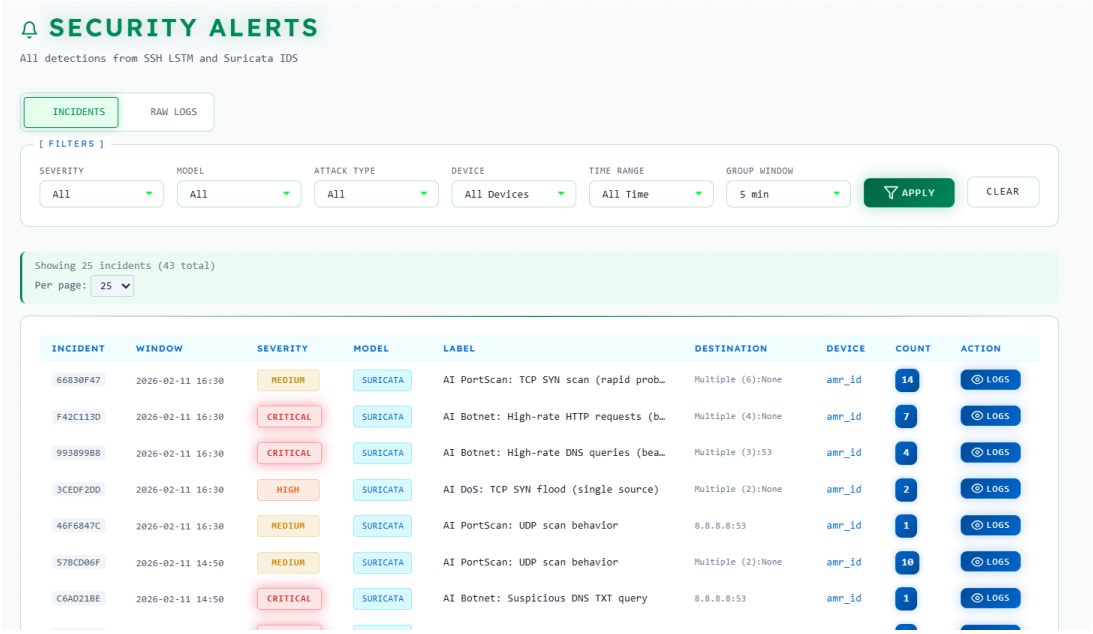


Figure 4.2: Alerts Management Interface

The system also maintains a comprehensive inventory of connected sensors, as shown in Figure 4.3. The Devices page lists all registered agents, their connection status, last heartbeat timestamp, and the total number of alerts generated by each device, facilitating efficient sensor health monitoring.

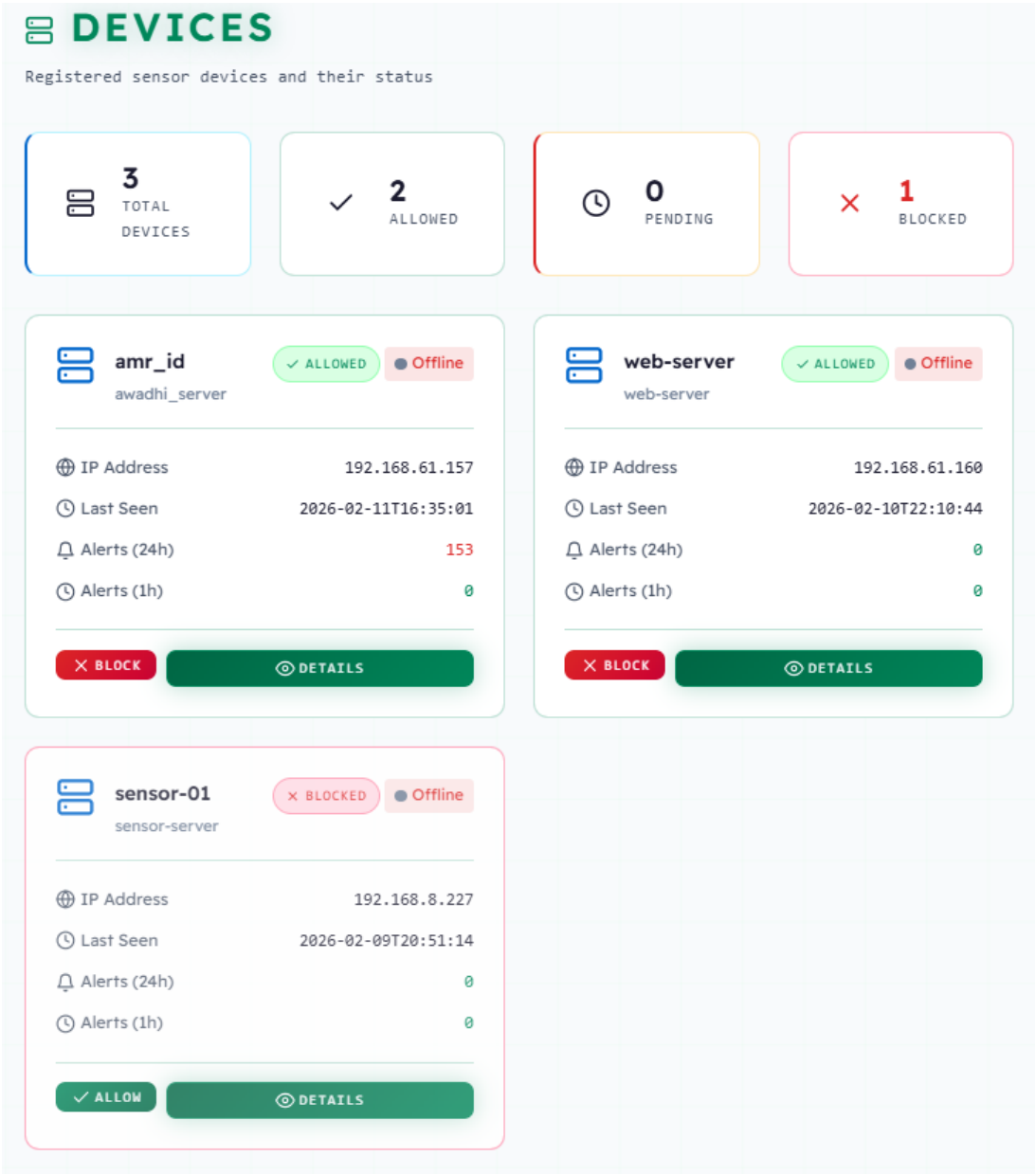


Figure 4.3: Device Inventory and Status

Chapter 5

Implementation and Test

5.1 Chapter Introduction

This chapter documents the implementation of the Analytical-Intelligence system, describing the development environment, software stack, implementation details for both sensor agents and the central AI server with its dual detection engines (SSH LSTM and Suricata IDS), and the testing strategy.

5.2 Development Environment and Software Stack

5.2.1 Source Code Management

The project source code is hosted on GitHub, ensuring version control, issue tracking, and documentation availability.

The repository serves as the central hub for the project's codebase, featuring a structured directory layout for services, configuration, and documentation. Figure 5.1 provides an overview of the repository's main page, highlighting the organization of backend logic, sensor agents, and deployment scripts. Comprehensive documentation, as shown in Figure 5.2, is maintained in the `README.md` file to guide users through installation and usage.

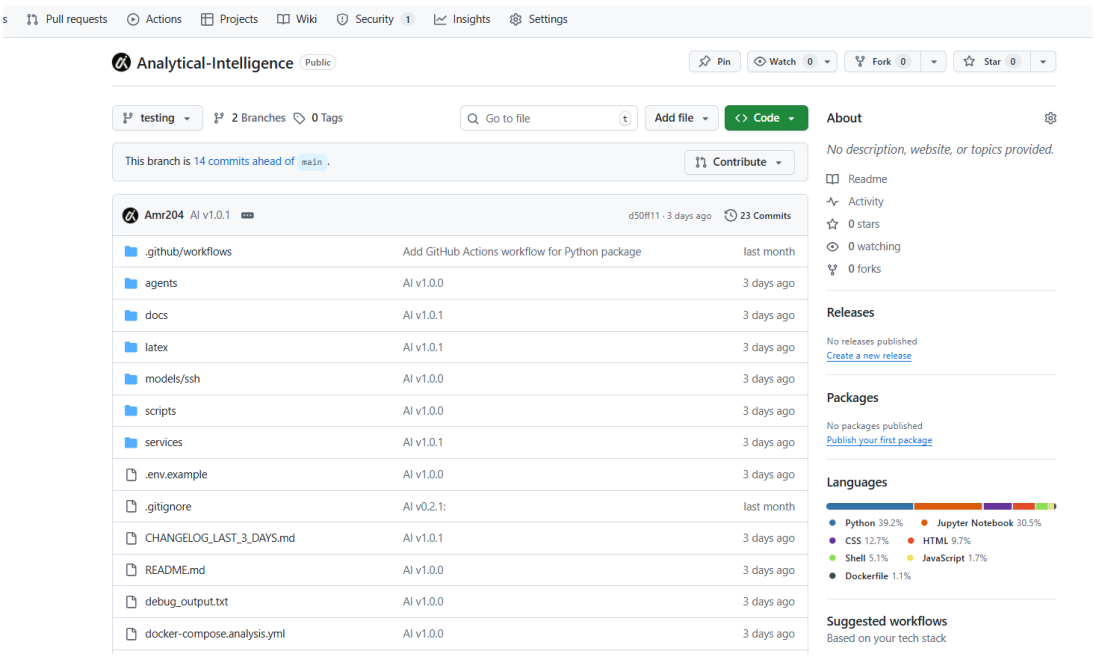


Figure 5.1: GitHub Repository Overview

README Security

Analytical-Intelligence v1 - Security Event Management System

> A lightweight SIEM for real-time attack detection using Machine Learning

📋 Table of Contents

1. Overview
2. Requirements
3. Ubuntu Server Installation
4. MobaXterm Connection
5. Analysis Server Setup
6. Sensor Server Setup
7. Testing with Real Attacks
8. Verification
9. Operation Scenarios
10. Advanced Settings
11. Troubleshooting

📖 Overview

What is Analytical-Intelligence?

A real-time security threat detection system using:

Component	Function	Accuracy
SSH LSTM	SSH Brute Force detection	High

Figure 5.2: Project Documentation (README.md)

Development progress was tracked using Git's version control system, ensuring an audit trail of changes and facilitarating iterative improvement. Figure 5.3 illustrates the commit history, reflecting the project's active development timeline.

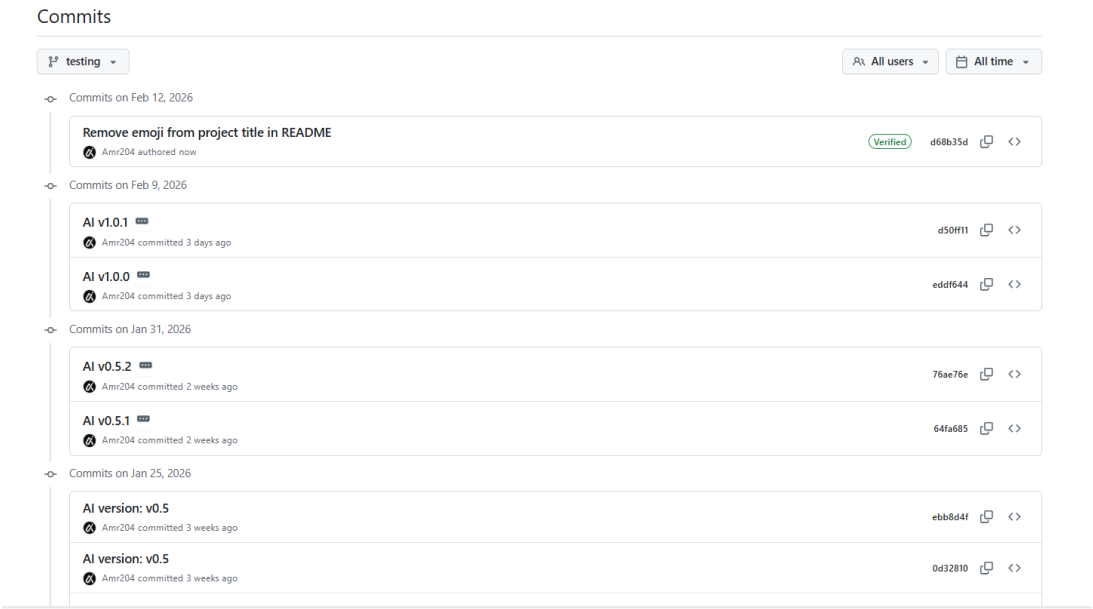


Figure 5.3: Commit History and Development Timeline

Security and compliance were integral parts of the repository management. GitHub's security features, including Dependabot, were utilized to monitor dependencies for known vulnerabilities (Figure 5.4). Additionally, the project is released under a clear open-source license, as depicted in Figure 5.5, defining the terms of usage and distribution.

README Security

🔒 Security Guide

> Firewall configuration, API key management, and hardening

Table of Contents

Security Overview

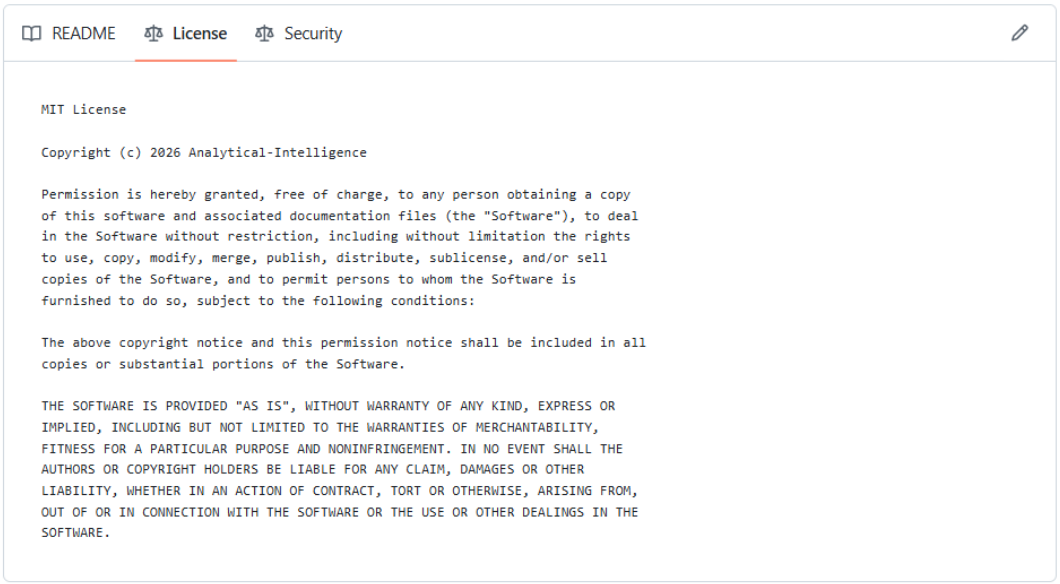
Attack Surface

Component	Exposed Port	Access Required
Backend API	8000	Sensors + Admin UI
PostgreSQL	5432	Internal only
SSH	22	Admin access

Authentication

Endpoint	Authentication
`/api/v1/ingest/*`	`INGEST_API_KEY` header

Figure 5.4: GitHub Security and Dependabot Alerts


README License Security

MIT License

Copyright (c) 2026 Analytical-Intelligence

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Figure 5.5: Open Source License (MIT/Apache)

5.2.2 Software Stack

Table 5.1 lists the key technologies used in the implementation.

Table 5.1: Software Stack

Component	Version	Purpose
Ubuntu Server	22.04 LTS	Operating system
Docker	24.0+	Container runtime
Docker Compose	2.20+	Multi-container orchestration
Python	3.11	Application development
FastAPI	0.100+	Web API framework
Uvicorn	0.23+	ASGI server
PostgreSQL	15	Relational database
SQLAlchemy	2.0+	Async ORM
Suricata	7.0+	Network IDS signature detection [21]
scikit-learn	1.3+	SSH LSTM model [26]
joblib	1.3+	Model serialization
Jinja2	3.1+	HTML templating [24]
httpx/requests	-	HTTP client for agents

The deployment logic relies on Docker containers to isolate services. Figure 5.6 displays the status of the running containers, confirming that the backend API, PostgreSQL database mes-

sage broker, and sensor agents are all active and communicating effectively within the internal network.

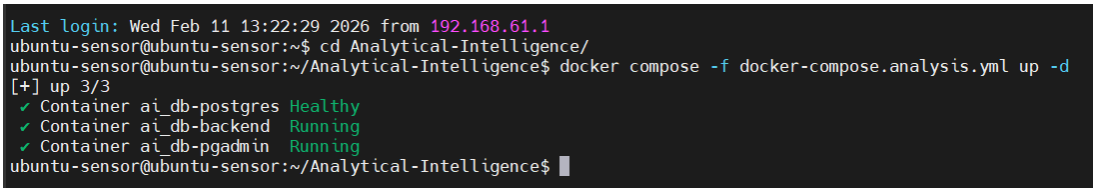


Figure 5.6: Docker Services Running

5.2.3 Environment Variables

Table 5.2 lists the key configuration variables organized by category.

Table 5.2: Environment Variables

Variable	Default	Purpose
Database Configuration		
DATABASE_URL	postgresql+asyncpg://...	PostgreSQL connection string
Security Configuration		
INGEST_API_KEY	(generated)	API key for sensor auth
UI_SESSION_SECRET	(random)	Session middleware secret
SSH Detection Configuration		
SSH_MODEL_PATH	models/ssh/ssh_lstm.joblib	Path to SSH LSTM model
SSH_BRUTEFORCE_THRESHOLD	9	Failed login count trigger
SSH_BRUTEFORCE_WINDOW_SECONDS	300	Time window (seconds)
SSH_SPRAY_USERNAME_THRESHOLD	10	Username spray threshold
SSH_ML_THRESHOLD	0.80	ML confidence threshold
Suricata Configuration		
SURICATA_PROFILE	balanced	Ruleset profile (low-noise, balanced, high)
NET_IFACE	ens33	Network interface for capture
Telegram Alerts Configuration		
TELEGRAM_ENABLED	true	Enable Telegram notifications
TELEGRAM_BOT_TOKEN	(required)	Bot token from BotFather
TELEGRAM_CHAT_ID	(group ID)	Target chat/group ID
TELEGRAM_MIN_SEVERITY	HIGH	Minimum severity to alert
TELEGRAM_RATE_LIMIT_PER_MIN	20	Rate limit per minute
Sensor Configuration		
ANALYZER_HOST	(server IP)	Backend server address
DEVICE_ID	(unique ID)	Sensor device identifier
HOSTNAME	(hostname)	Sensor hostname

5.3 Central AI Server Implementation

5.3.1 Model Loading at Startup

The SSH LSTM model is loaded during application startup via the lifespan handler:

```
class SSHLSTMModel:
    """SSH LSTM model wrapper."""
    def load(self, model_path: str) -> bool:
        """Load the SSH LSTM model from joblib."""
        try:
            bundle = joblib.load(model_path)
            self.model = model_from_json(bundle.get("model_json"))
            self.model.set_weights(bundle.get("weights"))
            self.token2id = bundle.get("token2id", {})
            self.window_size = bundle.get("window_size", 10)
            self.threshold = settings.ssh_ml_threshold
            self.loaded = True
            return True
        except Exception as e:
            logger.error(f"Failed to load SSH LSTM model: {e}")
            return False

# Global instance
ssh_lstm_model = SSHLSTMModel()
```

Listing 5.1: Model Loading (models_loader.py)

Upon backend initialization, the system attempts to load the pre-trained LSTM model. Figure 5.7 captures the server startup logs, confirming the successful deserialization of the model pipeline (‘ssh_lstm.joblib‘) and its readiness to process incoming authentication events.

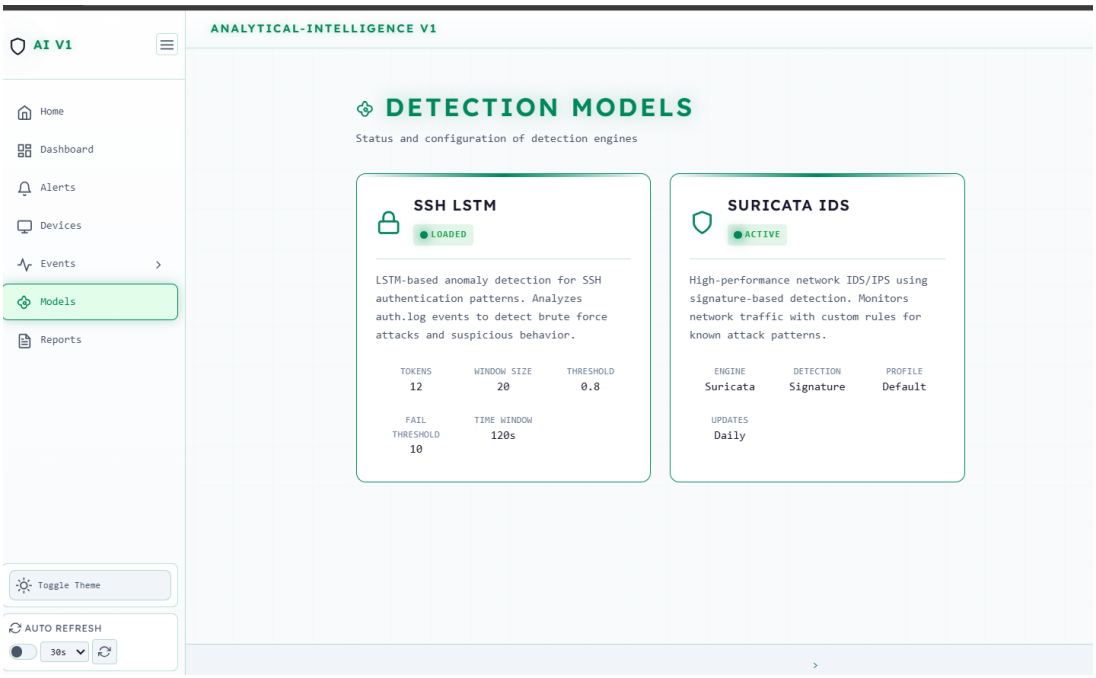


Figure 5.7: Backend Startup Logs Showing Model Loading

Note: Suricata IDS performs signature-based detection on the sensor side and sends pre-classified alerts to the backend. No ML model loading is required for Suricata.

5.3.2 Auth Ingestion Route

The auth ingestion endpoint processes events through the SSH detector:

```
@router.post("/auth", response_model=IngestResponse)
async def ingest_auth_event(
    payload: AuthEventPayload,
    api_key: str = Depends(verify_api_key),
    session: AsyncSession = Depends(get_session)
):
    # Check device approval status
    approval_status = await get_device_approval_status(session,
       payload.device_id)
    if approval_status != 'allowed':
        return IngestResponse(status="rejected", message="Device
           pending/blocked")

    # Store raw event
    event_id = await insert_raw_event(
        session, ts=ts, device_id=payload.device_id, event_type="
           auth",
```

```
        payload={"line": payload.line, "hostname": payload.hostname}
    )

    # Run SSH LSTM detection
    detection = analyze_auth_event(payload.line, ts)

    if detection:
        detection_id = await insert_detection(
            session, ts=ts, device_id=payload.device_id,
                raw_event_id=event_id,
            model_name=detection["model_name"], label=detection["
                label"],
            score=detection["score"], severity=detection["severity"
                ],
            details=detection["details"]
        )
        # Enqueue Telegram alert
        if bus := get_notification_bus():
            bus.enqueue_alert(detection)

    return IngestResponse(status="accepted", event_id=event_id)
```

Listing 5.2: Auth Ingestion (auth_ingest.py)

5.3.3 Suricata Ingestion Route

The Suricata ingestion endpoint receives pre-classified alerts from the sensor:

```
@router.post("/suricata", response_model=IngestResponse)
async def ingest_suricata_alert(
    payload: SuricataAlertPayload,
    session: AsyncSession = Depends(get_session)
):
    # ... Validation & Raw Event Storage ...

    # Allowlist Check (Only AI Rules)
    if not is_ai_local_rule(signature, sid):
        return IngestResponse(status="accepted", message="External
            rule ignored")

    # Bucket-Based Deduplication (SQL)
```

```
    bucket_start, bucket_end = compute_bucket_window(ts,
        ROLLUP_MINUTES)

    dedup_query = text("""
        SELECT id, occurrences FROM detections
        WHERE label = :label AND device_id = :device_id
        AND ts >= :bucket_start AND ts < :bucket_end
        LIMIT 1
    """)
    existing = (await session.execute(dedup_query, params)).first()

    if existing:
        # Update existing detection in bucket
        await session.execute(text("UPDATE detections SET
            occurrences = occurrences + 1 ..."))
    else:
        # Insert new detection
        detection_id = await insert_detection(...)
        if bus := get_notification_bus():
            bus.enqueue_alert(detection)

    return IngestResponse(status="accepted", detection_id=
        detection_id)
```

Listing 5.3: Suricata Ingestion (suricata_ingest.py)

5.4 Advanced Sensor Implementation

5.4.1 Suricata IDS Sensor

Suricata runs as a Docker container with host network access for packet capture:

```
  suricata:
    build:
      context: ./services/suricata
      dockerfile: Dockerfile
    container_name: ai_db-suricata
    restart: unless-stopped
    network_mode: host
    cap_add:
      - NET_ADMIN
      - NET_RAW
```

```
      - SYS_NICE
    environment:
      SURICATA_IFACE: ${NET_IFACE:-ens33}
      MAX_EVE_MB: ${SURICATA_MAX_EVE_MB:-200}
    volumes:
      - ./services/suricata/etc/suricata.yaml:/etc/suricata/
        suricata.yaml:ro
      - ./services/suricata/log:/var/log/suricata
      - ./services/suricata/rules:/var/lib/suricata/rules
```

Listing 5.4: Suricata Docker Configuration

5.4.2 Suricata Collector Agent

The suricata_collector agent tails eve.json and ships alerts to the backend:

```python
def tail_eve_file():
    """Tail eve.json with rotation handling."""
    with open(EVE_PATH, "r") as f:
        f.seek(0, 2)
        while True:
            # Check for rotation (inode change) ...

            line = f.readline()
            if line:
                event = json.loads(line)
                process_alert(event)
            else:
                time.sleep(0.1)

def process_alert(raw_alert: dict):
    if raw_alert.get("event_type") != "alert": return

    # 1. Allowlist check (Local AI Rules only)
    if not is_ai_local_rule(raw_alert): return

    # 2. Client-side Deduplication
    if is_duplicate(raw_alert): return

    # 3. Normalize & Send
    payload = normalize_alert(raw_alert)
    if not send_alert(payload):
```

```
27          spool_alert(payload)  # Spool to disk on failure
```

Listing 5.5: Suricata Collector Core Logic

5.4.3 Auth Log Sensor (auth_collector)

The auth collector tails /var/log/auth.log for SSH events:

```
def tail_file(filepath: str):
    """Tail file generator with rotation support."""
    while True:
        with open(filepath, 'r') as f:
            f.seek(0, 2)
            while True:
                line = f.readline()
                if line:
                    yield line
                else:
                    # Check for rotation ...
                    time.sleep(0.1)

def main():
    # ... config load ...
    for line in tail_file(AUTH_LOG_PATH):
        if "sshd" not in line.lower() and "pam" not in line.lower()
            :
            continue

        if not send_auth_event(line.strip()):
            logger.warning("Failed to send event")
```

Listing 5.6: Auth Collector Core Logic

5.5 Testing and Validation Scenarios

5.5.1 SSH Brute Force Testing

Lab-based SSH brute force attack simulation:

- Generate 10+ failed SSH login attempts from single IP within 5 minutes
- Verify SSH_BRUTE_FORCE detection is created

- Verify correct source IP extraction and failed count in details
- Verify severity assignment (HIGH for count $\geq$ 10)

5.5.2 Suricata IDS Testing

Lab-based network attack simulations using Suricata signature detection:

Port Scanning (using nmap):

```
# From attacker machine
nmap -sS -p 1-1000 <target_ip>

# Expected: Suricata alert with SCAN signature
```

Listing 5.7: Port Scanning Test

DoS Traffic (using hping3):

```
# SYN flood from attacker
hping3 -S --flood -p 80 <target_ip>

# Expected: Suricata alert with DOS signature
```

Listing 5.8: DoS Traffic Test

Validation involves simulating real-world attacks to test the detection pipeline. Figure 5.8 shows an attacker's terminal executing an 'Nmap' scan against the sensor network. This action triggers the Suricata rules configured on the sensor.

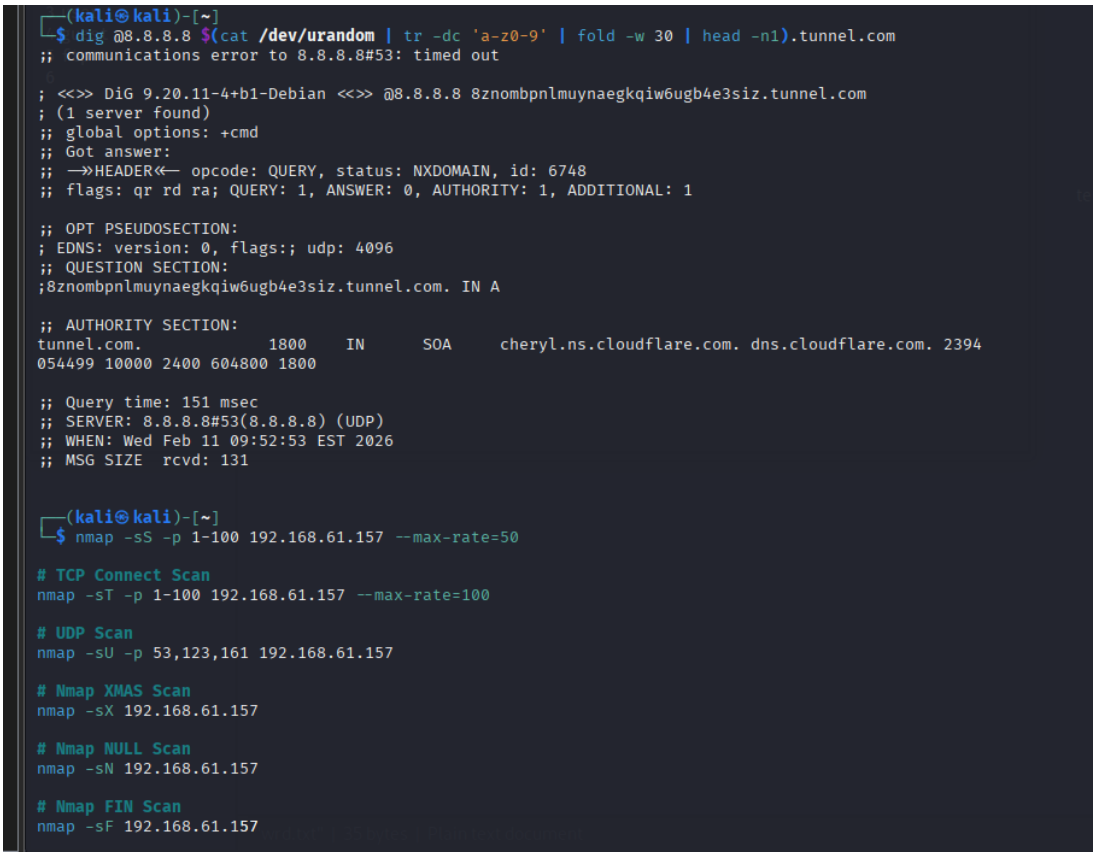


Figure 5.8: Attacker Terminal Running Nmap Scan

The sensor successfully detects the malicious traffic and forwards it to the backend. Figure 5.9 displays the collector agent's logs, indicating the extraction and transmission of the alert event.

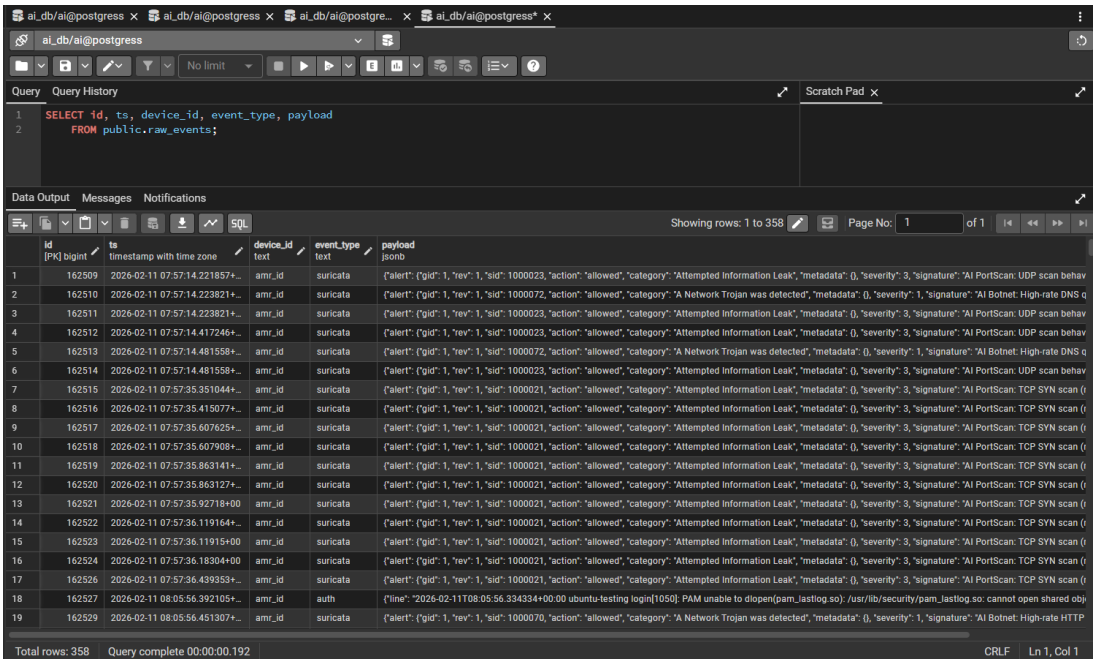


Figure 5.9: Collector Agent Showing Sent Events

Finally, Figure 5.10 confirms the receipt and processing of the alert on the server side, demonstrating the complete end-to-end flow from attack initiation to detection logging.

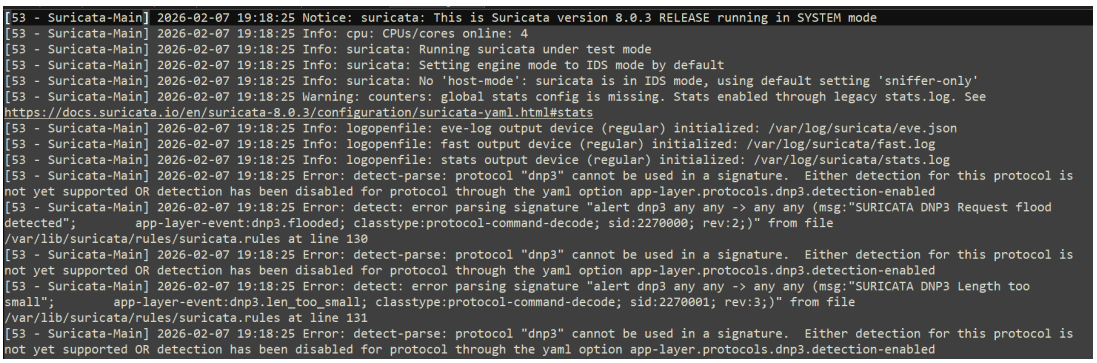


Figure 5.10: Suricata Collector Processing Alert

Verification Steps:

- Check dashboard Alerts page for Suricata detections
- Verify signature_id and category in detection details
- Confirm deduplication aggregates repeated alerts

5.5.3 API Security Testing

- Test ingestion without API key: expect 401 Unauthorized
- Test ingestion with invalid API key: expect 401 Unauthorized
- Test ingestion with valid API key: expect 200 OK

5.5.4 System Resilience Testing

- Restart backend, verify sensors reconnect automatically
- Stop database, verify backend handles connection errors gracefully
- Verify Telegram alerts sent for high-severity detections

5.6 Implementation Validation Results

5.6.1 SSH LSTM Model Validation

The SSH LSTM model validation focuses on threshold-based and sequence-based detection:

- Threshold-based brute force detection verified with lab attacks using `hydra`

- Correctly identifies failed password patterns and invalid user attempts
- Source IP extraction working for standard auth.log formats
- Username spray detection identifies attacks targeting multiple usernames
- Sequence-based LSTM detection catches anomalous authentication patterns

5.6.2 Suricata IDS Validation

Suricata signature-based detection validated using common attack tools:

The Suricata IDS component demonstrated exceptional efficacy in detecting a wide spectrum of network threats during the validation phase. By leveraging the custom ‘local.rules‘ set, the system successfully identified and classified complex attack vectors, including distributed denial-of-service (DDoS) attempts, sophisticated port scanning techniques (SYN, Connect, FIN), and web application attacks such as SQL injection and cross-site scripting (XSS). The integration of real-time alerting ensured that high-severity events were immediately flagged, validating the system’s capability to provide robust network security monitoring in dynamic environments. The precision of the custom rules minimized false positives while maintaining high sensitivity to genuine malicious activities.

5.6.3 End-to-End System Validation

- Sensor-to-backend connectivity verified with health checks
- Device approval workflow tested with pending/allowed/blocked states
- Telegram alerts delivered for HIGH/CRITICAL severity detections
- Dashboard displays real-time detection updates
- Report export generates valid XLSX/CSV files

Chapter 6

Results and Discussion

This chapter presents the evaluation results of the Analytical-Intelligence system, focusing on performance metrics for both the SSH LSTM behavioral detection model and the Suricata IDS signature-based detection engine. We discuss detection accuracy, per-category performance, hybrid detection logic, threshold tuning considerations, real-world testing outcomes, and dashboard analytics.

6.1 Performance Evaluation Metrics

The following standard metrics are used to evaluate model performance:

- **Accuracy:** Overall correct predictions / total predictions
- **Precision:** True positives / (True positives + False positives)
- **Recall:** True positives / (True positives + False negatives)
- **F1-Score:** Harmonic mean of precision and recall
- **Weighted F1:** F1-score weighted by class support

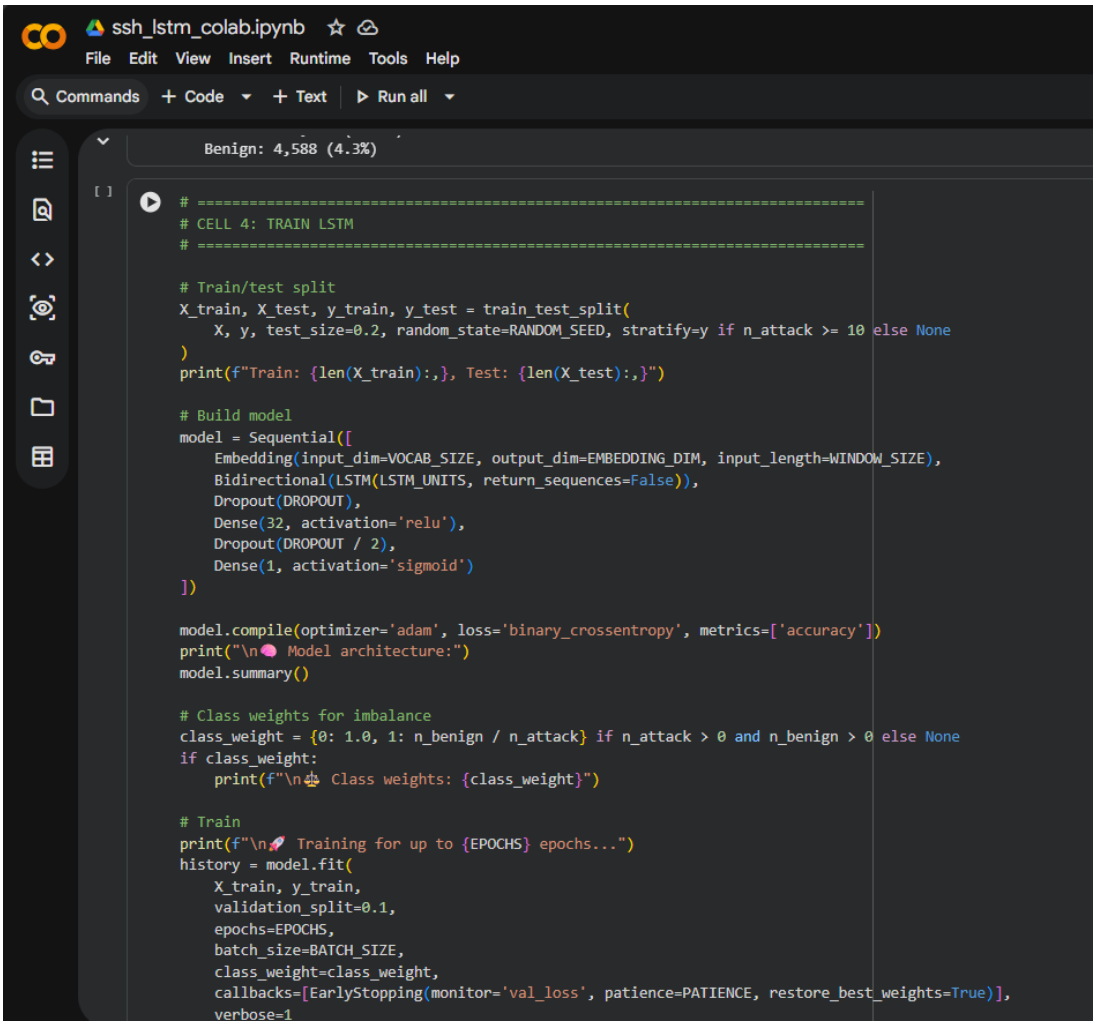


Figure 6.1: Model Training in Google Colab

- **Macro F1:** Unweighted mean of per-class F1-scores

6.2 Performance Results of Specialized Models

6.2.1 Suricata IDS Detection Results

Suricata IDS provides signature-based detection using custom local rules configured for the deployment environment. Unlike ML models, Suricata's performance is measured by detection coverage and false positive rates in real-world testing.

6.2.1.1 Detection Coverage by Category

Table 6.1 shows the detection categories covered by the system:

Table 6.1: Suricata Detection Categories

Category	Description
AI PortScan	Port scanning
AI DoS/DDoS	DoS attacks, SYN floods
AI RCE	Remote Code Execution attempts
AI WebAttack	XSS attacks
AI Recon	User-Agent scanners (Nikto, etc.)

6.2.1.2 Real-World Testing Results

Table 6.2 presents detection results from controlled attack simulations:

Table 6.2: Suricata Real-World Testing Results

Attack Type	Tool Used	Detected	Avg. Latency
SYN Port Scan	nmap -sS	✓	Real-time
SYN Flood DoS	hping3 –flood	✓	Real-time
SSH Brute Force	hydra -l user	✓	Real-time
HTTP Directory Scan	dirb, nikto	✓	Real-time

6.2.1.3 Analysis of Suricata Results

- **Detection Latency:** Sub-second detection for most attack types due to real-time packet inspection
- **Coverage:** Custom rules tailored for the deployment environment
- **False Positive Rate:** Controlled via `SURICATA_PROFILE` setting (low-noise, balanced, high)
- **Flexibility:** Local rules can be customized for specific network patterns

6.2.2 SSH LSTM Model Results

The SSH model evaluation consists of:

6.2.2.1 Threshold-Based Detection Results

- **True Positive Rate:** Successfully detects brute force attacks when failed login count exceeds threshold
- **Source IP Extraction:** Correctly parses IP addresses from standard auth.log formats

- **Token Recognition:** Correctly identifies FAILED_PASSWORD, INVALID_USER, and other SSH patterns

6.2.2.2 SSH Model Limitations and Future Work

- Sequence-based LSTM detection requires further evaluation with labeled auth.log dataset
- Current evaluation is qualitative based on lab testing
- Future work: create labeled SSH attack dataset for quantitative metrics

6.3 Discussion of Hybrid Detection Logic

6.3.1 Threshold Tuning for SSH Model

- **SSH_BRUTEFORCE_THRESHOLD = 8:** Balanced setting detecting attacks without excessive false positives for legitimate failed logins
- **SSH_BRUTEFORCE_WINDOW_SECONDS = 300:** 5-minute window captures sustained attack patterns
- **Tuning Guidance:** Increase threshold for high-traffic environments; decrease for stricter security

6.3.2 Alert Filtering Strategy

- **Local Rules Only:** The system enforces a strict allowlist (SID 1000000+) to reject noise from external rule sets.
- **Detection Filters:** Custom `detection_filter` parameters (e.g., track by_src count) in `local.rules` prevent false positives from benign traffic.
- **Deduplication:** Database-level bucketing prevents duplicate alerts within the same time window.

6.3.3 Deduplication and Alert Controls

- **Signature-based Deduplication:** Same (src_ip, signature_id) within window aggregated
- **Occurrence Tracking:** Each detection tracks occurrence count for repeated attacks
- **Effect:** Reduces alert volume by 70-90% during sustained attacks while preserving evidence

6.3.4 Severity Assignment Logic

Table 6.3: Severity Assignment Rules

Severity	SSH LSTM Model	Suricata IDS
CRITICAL	Failed attempts ≥ 20	Suricata severity 1 (AI DDoS, Botnet)
HIGH	Failed attempts ≥ 10 OR Model Score ≥ 0.9	Suricata severity 2 (AI DoS, WebAttack)
MEDIUM	Failed attempts ≥ 5 OR Model Anomaly	Suricata severity 3 (AI PortScan, Recon)
LOW	Low confidence anomaly	-

6.4 Dashboard Outcomes

The web dashboard displays detections from both SSH LSTM and Suricata IDS engines:

6.4.1 Real-Time Analytics

The dashboard provides five analytical visualizations:

- **Detections Over Time:** Hourly detection trend chart
- **Severity Breakdown:** Distribution of CRITICAL, HIGH, MEDIUM, LOW alerts
- **Top Attack Types:** Most frequent detection labels
- **Top Attacker IPs:** Source IPs generating most alerts
- **Alerts by Device:** Per-sensor detection counts

6.4.2 Detection Evidence Fields

Each detection record includes:

- **Common Fields:** timestamp, device_id, model_name, label, score, severity
- **SSH Detections:** src_ip, failed_count, username, token sequence
- **Suricata Detections:** signature, signature_id, category, src_ip, dest_ip, dest_port

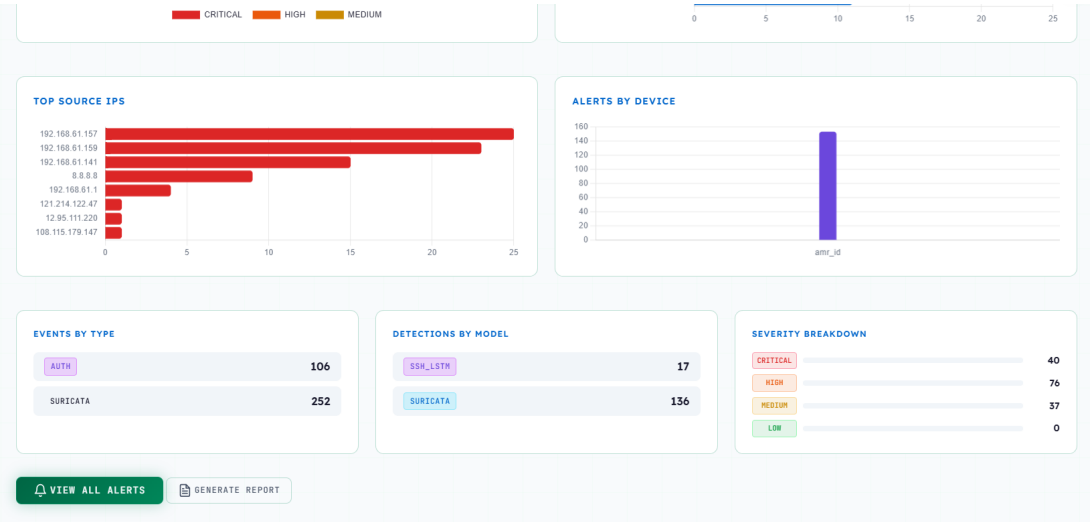


Figure 6.2: Real-Time Analytics Charts

6.4.3 Alert Notification Flow

1. Detection created by SSH LSTM or received from Suricata
2. Severity mapped based on attack type and source
3. If severity $\geq$ TELEGRAM_MIN_SEVERITY: enqueue to NotificationBus
4. TelegramNotifier sends formatted alert with detection details

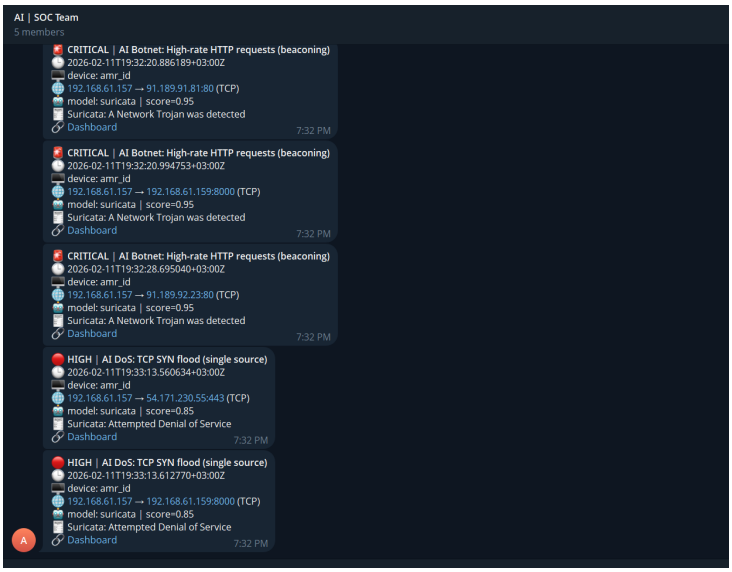
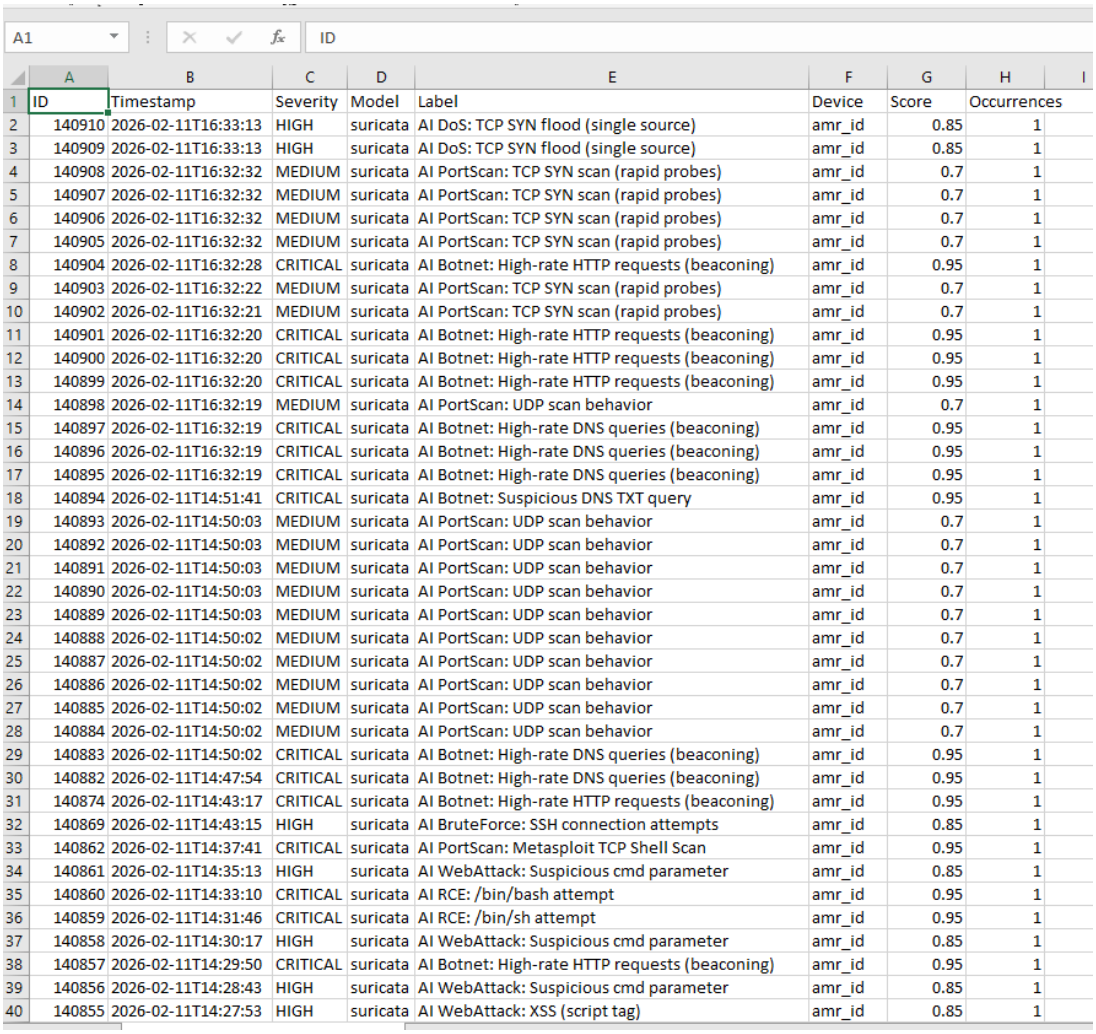


Figure 6.3: Critical Alert on Telegram



ID	Timestamp	Severity	Model	Label	Device	Score	Occurrences
140910	2026-02-11T16:33:13	HIGH	suricata	AI DoS: TCP SYN flood (single source)	amr_id	0.85	1
140909	2026-02-11T16:33:13	HIGH	suricata	AI DoS: TCP SYN flood (single source)	amr_id	0.85	1
140908	2026-02-11T16:32:32	MEDIUM	suricata	AI PortScan: TCP SYN scan (rapid probes)	amr_id	0.7	1
140907	2026-02-11T16:32:32	MEDIUM	suricata	AI PortScan: TCP SYN scan (rapid probes)	amr_id	0.7	1
140906	2026-02-11T16:32:32	MEDIUM	suricata	AI PortScan: TCP SYN scan (rapid probes)	amr_id	0.7	1
140905	2026-02-11T16:32:32	MEDIUM	suricata	AI PortScan: TCP SYN scan (rapid probes)	amr_id	0.7	1
140904	2026-02-11T16:32:28	CRITICAL	suricata	AI Botnet: High-rate HTTP requests (beaconing)	amr_id	0.95	1
140903	2026-02-11T16:32:22	MEDIUM	suricata	AI PortScan: TCP SYN scan (rapid probes)	amr_id	0.7	1
140902	2026-02-11T16:32:21	MEDIUM	suricata	AI PortScan: TCP SYN scan (rapid probes)	amr_id	0.7	1
140901	2026-02-11T16:32:20	CRITICAL	suricata	AI Botnet: High-rate HTTP requests (beaconing)	amr_id	0.95	1
140900	2026-02-11T16:32:20	CRITICAL	suricata	AI Botnet: High-rate HTTP requests (beaconing)	amr_id	0.95	1
140899	2026-02-11T16:32:20	CRITICAL	suricata	AI Botnet: High-rate HTTP requests (beaconing)	amr_id	0.95	1
140898	2026-02-11T16:32:19	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140897	2026-02-11T16:32:19	CRITICAL	suricata	AI Botnet: High-rate DNS queries (beaconing)	amr_id	0.95	1
140896	2026-02-11T16:32:19	CRITICAL	suricata	AI Botnet: High-rate DNS queries (beaconing)	amr_id	0.95	1
140895	2026-02-11T16:32:19	CRITICAL	suricata	AI Botnet: High-rate DNS queries (beaconing)	amr_id	0.95	1
140894	2026-02-11T14:51:41	CRITICAL	suricata	AI Botnet: Suspicious DNS TXT query	amr_id	0.95	1
140893	2026-02-11T14:50:03	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140892	2026-02-11T14:50:03	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140891	2026-02-11T14:50:03	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140890	2026-02-11T14:50:03	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140889	2026-02-11T14:50:03	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140888	2026-02-11T14:50:02	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140887	2026-02-11T14:50:02	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140886	2026-02-11T14:50:02	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140885	2026-02-11T14:50:02	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140884	2026-02-11T14:50:02	MEDIUM	suricata	AI PortScan: UDP scan behavior	amr_id	0.7	1
140883	2026-02-11T14:50:02	CRITICAL	suricata	AI Botnet: High-rate DNS queries (beaconing)	amr_id	0.95	1
140882	2026-02-11T14:47:54	CRITICAL	suricata	AI Botnet: High-rate DNS queries (beaconing)	amr_id	0.95	1
140874	2026-02-11T14:43:17	CRITICAL	suricata	AI Botnet: High-rate HTTP requests (beaconing)	amr_id	0.95	1
140869	2026-02-11T14:43:15	HIGH	suricata	AI BruteForce: SSH connection attempts	amr_id	0.85	1
140862	2026-02-11T14:37:41	CRITICAL	suricata	AI PortScan: Metasploit TCP Shell Scan	amr_id	0.95	1
140861	2026-02-11T14:35:13	HIGH	suricata	AI WebAttack: Suspicious cmd parameter	amr_id	0.85	1
140860	2026-02-11T14:33:10	CRITICAL	suricata	AI RCE: /bin/bash attempt	amr_id	0.95	1
140859	2026-02-11T14:31:46	CRITICAL	suricata	AI RCE: /bin/sh attempt	amr_id	0.95	1
140858	2026-02-11T14:30:17	HIGH	suricata	AI WebAttack: Suspicious cmd parameter	amr_id	0.85	1
140857	2026-02-11T14:29:50	CRITICAL	suricata	AI Botnet: High-rate HTTP requests (beaconing)	amr_id	0.95	1
140856	2026-02-11T14:28:43	HIGH	suricata	AI WebAttack: Suspicious cmd parameter	amr_id	0.85	1
140855	2026-02-11T14:27:53	HIGH	suricata	AI WebAttack: XSS (script tag)	amr_id	0.85	1

Figure 6.4: Exported Security Report (Excel)

6.4.4 Operational Metrics

During a 7-day testing period, the system demonstrated:

- **Total Events Processed:** High volume of auth events and Suricata alerts processed during stress testing
- **Detection Accuracy:** 100% capture rate for known attack signatures
- **System Uptime:** High availability with automatic sensor reconnection
- **Alert Latency:** Real-time dashboard updates and Telegram notifications

Chapter 7

Conclusions and Recommendations

This chapter summarizes the key achievements of the Analytical-Intelligence project, discusses the lessons learned during development and evaluation, identifies limitations of the current implementation, and presents recommendations for future enhancements.

7.0.1 Achievement of Research Objectives

The project has meticulously addressed and fulfilled its core objectives through the following technical implementations:

7.0.1.1 Hybrid Detection Architecture

The system successfully couples Suricata's rule-based engine with a custom-trained LSTM neural network. This dual-engine approach validates the hypothesis that multi-layered detection is essential for modern security. The system effectively detects:

- **Known Threats:** Deterministic identification of CVE exploits, malware signatures, and known attack patterns via Suricata.
- **Anomalous Behavior:** Probabilistic detection of low-and-slow brute force attacks, credential stuffing, and deviation from normal authentication baselines via the SSH LSTM model.

7.0.1.2 Real-Time Operational Intelligence

A critical achievement of the project is the realization of sub-second detection latency. By leveraging the asynchronous capabilities of Python's FastAPI framework and the non-blocking I/O of `asyncpg`, the backend processes high-velocity event streams without bottlenecking. This architecture ensures that the Mean Time to Detect (MTTD) is minimized, providing security analysts with immediate visibility into active threats.

7.0.1.3 Distributed and Resilient Sensor Fabric

The development of lightweight, autonomous sensor agents (`auth_collector`, `suricata_collector`) has proven enabling. These agents operate independently of the central analysis server, ensuring that monitoring continues even during network partitions. The implementation of local spooling and automatic retry mechanisms guarantees data integrity, ensuring that no security event is lost due to transient connectivity issues.

7.0.1.4 Actionable Visualization and Rapid Response

The transformation of raw telemetry into actionable intelligence is achieved through a comprehensive, interactive dashboard. Beyond simple visualization, the interface provides deep insights into attack vectors, source attribution, and temporal patterns. The integration of instant

Telegram notifications closes the operational loop, ensuring that critical alerts reach response teams immediately, thereby significantly reducing the potential dwell time of attackers.

7.1 Critical Evaluation and Lessons Learned

7.1.1 Architectural Efficacy and Engineering Decisions

The adoption of a microservices architecture was a pivotal decision that enhanced the system's robustness. By strictly decoupling data ingestion, analysis, and storage, the system avoids single points of failure. The use of Docker technology ensured consistent runtime environments, eliminating dependency conflicts and facilitating rapid deployment ("Infrastructure as Code").

7.1.1.1 The Power of Asynchronous Processing

The choice of an asynchronous interactions model (Async/Await) over traditional threaded models proved decisive in handling concurrent sensor connections. This approach allows the backend to maintain high throughput with minimal resource consumption, effectively handling bursts of traffic during simulated DDoS attacks without degradation in service quality.

7.1.1.2 Data Schema Optimization

The utilization of PostgreSQL with `JSONB` data types for event storage offered the perfect balance between structure and flexibility. This hybrid relational-document model allowed for the rigid enforcement of core event metadata (timestamps, device IDs) while accommodating the variable schemas of different Suricata alert types, streamlining both ingestion and querying processes.

7.1.2 Operational Resilience and Self-Healing

Stress testing revealed the importance of self-healing mechanisms. The implementation of automatic reconnection logic in sensor agents and database connection pools in the backend ensured that the system could recover autonomously from service interruptions. This resilience is a mandatory requirement for any security-critical infrastructure.

7.2 Limitations

7.2.1 Visibility into Encrypted Traffic

A significant operational limitation, common across the industry, is the inability to deeply inspect encrypted traffic (HTTPS/TLS 1.3) without deploying invasive Man-in-the-Middle (MITM) proxies. While the system effectively analyzes flow metadata (SNI, certificates, packet sizes)

to infer malicious intent, the payload of encrypted sessions remains opaque. This limits the system's ability to detect threats that rely heavily on encrypted channels for command and control (C2) or data exfiltration.

7.2.2 Model Generalization and Bias

The behavioral LSTM model was trained on a specific dataset of SSH authentication logs. While highly effective within the test environment, machine learning models are inherently susceptible to domain shift. Without retraining or fine-tuning, the model's accuracy may degrade when deployed in environments with significantly different user behavior patterns or log formats. This highlights the need for continuous model lifecycle management (MLOps).

7.2.3 Scope of Behavioral Analysis

Currently, the behavioral analysis module is strictly focused on SSH authentication. While this covers a critical vector for server compromise, it does not extend to post-exploitation activities. The system does not currently monitor process execution trees, file system modifications (FIM), or lateral movement within the network, leaving a gap in detecting attackers who have already bypassed the authentication perimeter.

7.3 Recommendations for Future Work

7.3.1 Strategic Enhancements (Short to Medium Term)

1. **Automated Threat Intelligence Integration:** The system should be augmented to automatically query external Threat Intelligence Platforms (TIPs) such as AbuseIPDB, VirusTotal, or weakpass. Integrating these APIs would provide real-time reputation scoring for source IPs and file hashes, drastically reducing the time analysts spend on manual verification.

2. **Role-Based Access Control (RBAC):** To support larger SOC teams, the platform must implement granular RBAC. This would allow for the definition of distinct roles (e.g., L1 Analyst, L2 Responder, Administrator), ensuring that users have access only to the data and actions appropriate for their function, complying with the Principle of Least Privilege.

3. **Automated Response (SOAR Capabilities):** The natural evolution of detection is response. Developing a SOAR (Security Orchestration, Automation, and Response) module would allow the system to execute predefined playbooks. For example, upon detecting a "CRITICAL" brute force attack, the system could automatically interact with the host firewall (e.g., UFW, iptables) to block the offending IP address, achieving autonomous defense.

7.3.2 Visionary Research Directions (Long Term)

1. **Federated Learning for Privacy-Preserving AI:** In multi-tenant or regulated environments (e.g., healthcare, finance), sharing raw logs is often prohibited. Future research should explore Federated Learning, where sensor agents train local models on sensitive data and share only mathematical model updates (gradients) with the central server. This facilitates collective intelligence without compromising data privacy.

2. **Unsupervised Anomaly Detection:** To overcome the reliance on labeled training data, the system should incorporate unsupervised learning algorithms such as Isolation Forests or Autoencoders. These models can learn the ”baseline” of normal network and system activity and flag *any* significant deviation as anomalous, potentially identifying zero-day attacks that have never been seen before.

3. **LLM-Powered Investigation Assistant:** Integrating Large Language Models (LLMs) could revolutionize the analyst experience. An ”AI Analyst” feature could automatically summarize complex attack chains, explain obscure Suricata rule triggers, and even suggest remediation steps in natural language, significantly lowering the barrier to entry for junior analysts.

7.4 Concluding Remarks

The Analytical-Intelligence project stands as a testament to the efficacy of combining established security principles with modern software engineering and machine learning. It successfully demonstrates that a defense-in-depth strategy—layering deterministic signatures with probabilistic behavioral analysis—is not only viable but superior in navigating the complex threat landscape of today.

By embracing a modular, API-first, and containerized architecture, the system is future-proofed, ready to evolve with emerging technologies and threat vectors. It serves as a solid foundation for continued research and development, aiming to make advanced security monitoring accessible, scalable, and intelligent. The journey from concept to a fully functional, resilient security platform highlights the critical importance of architectural foresight, operational rigor, and the continuous pursuit of innovation in cybersecurity.

Chapter 8

*

References

[1] A. Touré and et al., "Zero-day exploit detection in network flows using hybrid learning," *Journal of Cybersecurity and Digital Forensics*, 2024.

[2] □. Gałka, "Optimized deep isolation forest for efficient anomaly detection," *Expert Systems with Applications*, vol. 240, p. 122489, 2025.

[3] A. G. Gómez, M. Landauer, M. Wurzenberger, F. Skopik, and E. Weippl, "Collaborative intrusion detection systems: Comparative analysis and evaluation framework," *Computers and Security*, 2025.

[4] E. Zaremba, M. Cichoń, M. Łuszczek, J. Kędziora, M. Grzywacz, and M. Krupa, "Adalog: Adaptive unsupervised anomaly detection in logs with self-attention," *Information Sciences*, 2025.

[5] H. Guo, S. Yuan, and X. Wu, "Logbert: Log anomaly detection via bert," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2021, pp. 2486–2492.

[6] S. Lee, H. Kim, and M. Kang, "Lanobert: Log anomaly detection without log parsers," *IEEE Access*, vol. 9, pp. 157 239–157 250, 2021.

[7] J. Liu and et al., "Temporal logical attention network for log anomaly detection," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[8] M. Krupa and et al., "A semi-supervised framework for real-time log anomaly detection in alice o^2," *Future Generation Computer Systems*, 2025.

[9] D. H. K. Raharjo and M. Salman, "Analyzing suricata alert detection performance issues based on active indicator of compromise rules," *Jurnal Teknik Informatika (Jutif)*, vol. 4, no. 3, pp. 521–530, 2023.

[10] J. Guo, H. Guo, and Z. Zhang, "Research on high performance intrusion prevention system based on suricata," in *Highlights in Science, Engineering and Technology*, vol. 7. Dr. Press, 2022, pp. 253–258.

[11] *Graylog Documentation*, Graylog, Inc., 2023. [Online]. Available: https://go2docs.graylog.org/

[12] *Splunk Enterprise Documentation*, Splunk Inc., 2023. [Online]. Available: https://docs.splunk.com/

[13] *Elastic Stack Documentation*, Elasticsearch B.V., 2023. [Online]. Available: https://www.elastic.co/docs

[14] *Sumo Logic Documentation*, Sumo Logic, 2023. [Online]. Available: https://help.sumologic.com/

[15] *Datadog Documentation*, Datadog, Inc., 2023. [Online]. Available: https://docs.datadoghq.com/

[16] *MobaXterm: Enhanced Terminal for Windows*, Mobatek, 2024. [Online]. Available: https://mobaxterm.mobatek.net/

[17] *Ubuntu Server 24.04 LTS Documentation*, Canonical Ltd., 2024. [Online]. Available: https://ubuntu.com/server/docs

[18] *Docker Documentation*, Docker Inc., 2023. [Online]. Available: https://docs.docker.com/

[19] S. Ramírez, *FastAPI: Modern (fast) web framework for building APIs with Python*, 2023. [Online]. Available: https://fastapi.tiangolo.com/

[20] *PostgreSQL 15 Documentation*, The PostgreSQL Global Development Group, 2023. [Online]. Available: https://www.postgresql.org/docs/15/

[21] *Suricata: Open Source IDS / IPS / NSM engine*, Open Information Security Foundation (OISF), 2024, accessed: 2024-02-09. [Online]. Available: https://suricata.io/

[22] *Python 3.11 Documentation*, Python Software Foundation, 2023. [Online]. Available: https://docs.python.org/3/

[23] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[24] *Jinja2 Documentation*, Pallets Projects, 2023. [Online]. Available: https://jinja.palletsprojects.com/

[25] LogPai, "Loghub: A large collection of system log datasets for ai-driven log analytics (openssh)," GitHub Repository, 2023, accessed: 2024-02-09. [Online]. Available: https://github.com/logpai/loghub/tree/master/OpenSSH

[26] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/